

VFIDE

Veritas + Fides — Truth + Trust

Complete Manual

A field guide for users, merchants, operators, and developers — covering everything from a first wallet connection to the deepest contract internals.

Version: 1.0 — Testnet Edition

Date: May 10, 2026

Codebase baseline: v19.13

Networks: Base Sepolia (testnet); Base, Polygon, zkSync Era (mainnet targets)

Why VFIDE Exists

Half the world has no bank account. The other half pays a tax for having one.

There are 1.4 billion adults on earth with no bank account at all. There are another two to three billion adults who do have access to financial services but pay disproportionately for them. That second group is the larger one — and it is not just the poor. It includes the small business owner in São Paulo watching three percent of every card sale flow to networks she has never met. It includes the freelance designer in Mumbai whose client in London pays through PayPal and loses four percent to fees, then another two percent to the currency spread. It includes the construction worker in Dubai who sends three hundred dollars home each month and watches Western Union take twenty-five to forty-five of those dollars. It includes the developer in Buenos Aires whose dollar earnings are converted to pesos at a rate the bank decides, then taxed when she tries to convert them back. It includes the coffee farmer in Guatemala whose cooperative promises fair prices but takes most of the margin before any of the money reaches her.

These people are not unbanked. They have accounts, cards, apps. They are *extracted from* — every transaction is a small leak, every transfer a small loss, and over a year the leaks add up to weeks of work taken without anything obvious in return. The financial system was built by the wealthy to serve the wealthy. Everyone else pays the cost of operating it.

How the extraction works

Banks charge the poor more than the rich for the same services. Minimum balance fees, overdraft charges, foreign transaction surcharges, and maintenance fees are all regressive — the dollar amounts are the same, but they represent a far larger share of income for someone earning two dollars a day than for someone earning two hundred. A ten-dollar monthly account fee on a two-hundred-dollar balance is a five percent annual tax on being poor. The banks know this. They do not change it because small accounts are cost centers, not profit centers.

Payment processors take from the merchant. Stripe charges 2.9% plus thirty cents per transaction. PayPal charges 3.49% plus forty-nine cents. Square charges 2.6% plus ten cents. These rates do not decrease regardless of volume, loyalty, or risk profile. A merchant who has processed ten thousand successful transactions with zero chargebacks pays the same percentage as a brand new account. There is no mechanism to reward trustworthiness because the business model depends on taking a fixed percentage of everything. The cost is either absorbed by the merchant — reducing the margin on actual work — or passed to the customer through hidden price increases. Either way the buyer and seller both lose, and the processor wins on every transaction either of them ever makes.

Mobile money improved access but built new extraction. M-Pesa and its equivalents brought basic financial services to hundreds of millions of people — a genuine achievement. But the fee structures are regressive. Sending ten dollars costs one to three percent. Withdrawing ten dollars costs another one to three percent. The fees are flat or nearly flat, which means small transactions pay disproportionately more. A person moving two hundred dollars per month through M-Pesa loses four to eight dollars per month in fees — which may be a full day's income. There is no path to lower fees. No

loyalty benefit. No reputation that earns better rates. You pay the same on day one as on day one thousand.

Cross-border remittances are the worst form of all. A construction worker in the Gulf states sending three hundred dollars home pays Western Union or MoneyGram between eight and fifteen percent. The money takes three to five business days to arrive. There is no version of "I am a reliable sender" that gets him a better rate. He cannot accumulate reputation. He pays the immigrant tax forever.

The middle class pays in slower, quieter ways. Foreign-currency spreads on debit cards. Wire fees both directions on international transfers. Stablecoin off-ramps that quote you a rate three percent worse than the wholesale rate and call it a service. Subscription tools that extract a piece of every customer transaction (Stripe Connect, Square for Restaurants, Shopify Payments). Card networks that take their cut, processors that take their cut, banks that take their cut, payment apps that take their cut. By the time a five-dollar coffee reaches the barista's actual wages, half a dozen institutions you have never met have taken something.

Crypto promised inclusion and delivered speculation. Bitcoin was introduced as peer-to-peer electronic cash. It became a speculative asset traded by hedge funds. DeFi was supposed to democratize finance. It became yield farming for people who already had six-figure portfolios. NFTs were supposed to empower creators. They became a casino. Every wave of crypto innovation begins with "banking the unbanked" in the pitch deck and ends with "optimizing APY for whales" in the product. The fundamental problem: crypto wallets require understanding seed phrases, private keys, and network selection. Losing a phone with a mobile wallet means losing everything — permanently. There is no customer service number, no recovery process, no safety net. The unbanked person who finally saved two hundred dollars and put it in a crypto wallet has less protection than she had under the mattress.

The common thread

In every case, the people who do the actual work — growing food, making clothing, providing services, building software, building houses, sending money home, running small businesses — are the ones who pay the most and receive the least. The system is not broken. It is working exactly as designed. It was designed to move value upward.

VFIDE was built to move it back.

What VFIDE does differently

VFIDE is a payment protocol built on three commitments. First, the merchant receives every cent of the listed price. The buyer pays the network fee, separately and visibly. The cost of operating the system falls on the user of the system, not the provider of the goods. Second, trust is earned through behavior, not granted by authority — and earned trust lowers fees, raises limits, and unlocks features. A new user pays around 3.82% on each transfer; a trusted user with a track record pays 0.25%. Third, no entity — not the developer, not the elected council, not any single user — can freeze, blacklist, or seize anyone else's tokens. The functions that would have enabled it are permanently absent from the contract code.

These commitments are not promises. They are encoded in 108 smart contract files. The merchant fee is hardcoded to zero. The trust-curve is an immutable linear interpolation between two thresholds. The freeze function does not exist on the post-handover admin panel — not as a removed feature, but as a deliberate absence of code. The protocol cannot do these things to you because the code that would do them was never written.

VFIDE serves the unbanked because the unbanked need a place to keep value that does not require trust in institutions. It serves the underbanked because the underbanked are tired of paying a tax for the privilege of being trusted by an institution that was always extracting from them. It serves the global middle class because the middle class is tired of subsidizing every transaction with a quiet, regressive cut to networks they never chose. It serves freelancers, remote workers, immigrants, small business owners, merchants, farmers, and anyone whose life involves moving money — which is to say, everyone.

The system was built by the powerful for the powerful. VFIDE was built for everyone they extracted from.

Table of Contents

Why VFIDE Exists	2
Front matter	
Reading guide — who this is for and how to use it	7
60-second summary	8
Part I — Quick Start	9
Adwoa's first ten minutes	10
Power-user quickstart	13
Mini glossary — 10 terms to know	16
Part II — User Manual	17
Chapter 1 · Wallets, Vaults, and You	19
Chapter 2 · Sending and Receiving Payments	24
Chapter 3 · ProofScore — Your Trust Number	29
Chapter 4 · Guardians and Recovery	35
Chapter 5 · For Merchants	40
Chapter 6 · Governance — Voting and the Council	45
Chapter 7 · Endorsements and the Social Layer	51
Chapter 8 · Lending and Borrowing	54
Chapter 9 · Badges and Achievements	58
Chapter 10 · Disputes, Fraud Reports, and Appeals	63
Part IV — Technical Reference	67
T1 · Smart Contract Reference	68
T2 · API Reference	72
T3 · Database Schema	75
T4 · Configuration Reference	77
T5 · Security Model	79

T6 · Howey Compliance Notes	82
T7 · Test Coverage Map	84
Part V — Appendices	85
Appendix 1 · Glossary (130 entries)	86
Appendix 2 · Frequently Asked Questions	93
Appendix 3 · Troubleshooting	96
Appendix 4 · Reference Card (every threshold in one place)	99
Appendix 5 · Change Log + project history	105

Reading guide — who this is for and how to use it

This manual is written for two readers, side by side.

Adwoa is a fruit vendor in Kumasi. She has a smartphone and uses WhatsApp every day. She has heard the word "Bitcoin" and is not sure what it means. She wants to stop losing 3.5 percent of every sale to Visa, and she wants to send money to her cousin without Western Union taking eight percent. She does not want to learn what a blockchain is to do those things.

Marcus is a senior smart-contract auditor. He has skimmed the codebase and wants every threshold value, every event signature, every contract function in one place — with the line numbers in the source so he can verify everything for himself.

Both of them are reading the same book. Each chapter is layered.

How the layers work

N.1 In Plain English. Three to five short paragraphs. No code, no jargon. If Adwoa reads only the N.1 section of every chapter she still understands what VFIDE is and how to use it.

N.2 How It Works. The mechanism explained without code, often with a diagram. Why it is built this way, where the trade-offs are, what edge cases users will hit.

N.3 Technical Reference. Function signatures, exact constant values, error codes, line citations into the codebase. This is what Marcus opens to.

If you are Adwoa and the technical sections feel intimidating, skip them. They are there for someone else. If you are Marcus and the plain-English sections feel basic, scan the diagrams and skip to N.3. The book is designed to let both of you ignore what is not for you without missing anything you need.

How to find what you need

The table of contents lists every chapter. The glossary at the back defines every term used in the body. The index at the back maps every meaningful term to the page where it appears. If you do not know what a word means, look it up in the glossary first; if you want to read more about a topic, look it up in the index.

Code citations look like this: `VFIDEToken.sol:52`. That means file `contracts/VFIDEToken.sol`, line 52, in the v19.13 codebase. Anyone can verify any claim in this manual against the actual code.

60-second summary

VFIDE is a payment system for people the financial system has left out. Every user has a smart-contract *vault* that holds their tokens; their phone or wallet app is just a key. Lose the phone, the funds are still safe in the vault, and trusted guardians the user chose can help recover access.

Every user also has a *ProofScore* — a number from 0 to 10,000 that tracks honest behavior on-chain. Higher score, lower fees. A new user pays five percent on a transfer; a trusted user with a score of 8,000 pays one quarter of one percent. Trust replaces credit history.

The merchant always receives the full price. The buyer pays the small network fee, visible as a line item. There is no hidden processing cost flowing back through the merchant. The fee splits forty percent burned, ten percent to a community charity fund, fifty percent to ecosystem operations.

Governance is a twelve-member elected council whose votes are weighted by ProofScore, not by token balance. After six months of bootstrap, the developer's administrative key burns and the protocol becomes fully community-governed. Freezing a user's tokens or seizing them becomes architecturally impossible — the functions to do it are not in the code.

If you remember nothing else

VFIDE is non-custodial: the protocol cannot freeze, blacklist, or seize a user's tokens after the bootstrap period. Trust is earned by behavior, not granted by authority. The merchant pays no protocol fee.

Part I

Quick Start

Adwoa's first ten minutes

This section assumes you have a smartphone, an internet connection, and that you have never used a cryptocurrency wallet before. Follow these steps in order. When you finish, you will have a working VFIDE account, some test tokens to practice with, and at least one trusted guardian who can help you recover your account if your phone is ever lost or stolen.

Before you start

Pick a quiet ten minutes when you can read carefully. None of this is hard, but rushing makes mistakes. Have your phone in your hand and a pen and paper nearby — you will write down two things and keep them safe.

Step 1 — Install a wallet app

A wallet is the app on your phone that lets you connect to VFIDE. The wallet is like a bank card: it identifies you and authorizes payments, but it is not where your money lives. Your money lives in something called a *vault*, which we will set up automatically in step 4.

On Android or iPhone, install one of these apps from your normal app store. Any one of them will work:

- **MetaMask Mobile** — the most common option.
- **Rainbow** — friendly interface, also widely used.
- **Coinbase Wallet** — separate from the Coinbase exchange app, but made by the same company.

When you open the app for the first time, it will ask you to create a new wallet. Follow the steps in the app. It will show you a list of twelve words called a *recovery phrase*. Write these twelve words on the paper, in order, and put the paper somewhere very safe. **Do not type these words into any website. Do not photograph them. Do not send them in WhatsApp.** If anyone gets these twelve words, they can take everything in your wallet. If you lose them and forget them, no one can help you.

Why are the twelve words dangerous, and why do we still write them down?

The twelve words are the master key to your wallet app. They are dangerous because they bypass every other protection — anyone with the words can act as you. We still write them down because the wallet app might be deleted by accident, or the phone might break. The twelve words are how you get back in. VFIDE adds a second layer of safety — the vault and guardians, set up in steps 4 and 5 — so that even if someone steals the twelve words, they cannot drain everything immediately.

Step 2 — Open the VFIDE app

On the same phone, open your web browser and go to the VFIDE address your community contact gave you (during testnet, this will be a Sepolia testnet URL). The page will say something like *Connect Wallet*. Tap it. A pop-up from your wallet app will appear, asking permission for VFIDE to read your address. Tap *Connect*. VFIDE never sees your twelve words, never sees your private key. It only sees your public address — like seeing a postal address but never the inside of the house.

Step 3 — Get your first test VFIDE

Once connected, look for a button labelled *Faucet* or *Get test tokens*. Tap it. Sign the transaction your wallet pops up — there is no real cost on testnet, this is for practice. After about thirty seconds, you should see a small balance appear: typically 100 VFIDE for testing, plus a tiny amount of test ETH for transaction gas. These are not real money; testnet VFIDE has no value outside testing. The faucet works once per day per address.

Step 4 — Your vault is created automatically

The moment your first VFIDE arrives, the system deploys a smart-contract vault on-chain just for you. You do not have to do anything to make this happen — it is part of the receive-tokens transaction. From now on, your VFIDE lives inside the vault. Your wallet is the key to the vault, the way an ATM card is the key to a bank account.

The VFIDE app will display your vault address. Write this down too, on the same paper as your twelve words (it is fine for the vault address to be public — unlike the twelve words, you can share it). The vault address is how anyone sends VFIDE to you.

Step 5 — Add at least one guardian

A guardian is a trusted person — a relative, a neighbor, the shopkeeper next to your stall — whose own VFIDE address you list as someone allowed to help you recover your vault if your phone is lost. You need at least two guardians to do anything sensitive. We recommend three: your sister, your neighbor, and one more.

In the VFIDE app, find the *Guardians* page. Tap *Add guardian*. Enter that person's VFIDE address. They will need to confirm. Once you have at least two guardians and the system shows *Guardian setup complete*, you are protected. If your phone breaks, your guardians together can authorize a new key to take over your vault. No one of them alone can do it.

What if I do not know anyone else with VFIDE yet?

You can still use the system without guardians while you find people, but the protections are weaker. Until guardian setup is complete, you cannot perform sensitive operations like wallet rotation. The first thing to do after step 5 is help the people you trust set up their own VFIDE so they can be your guardians. The Headhunter Pool actually pays you for this — see chapter 5.

Step 6 — Make your first payment

Find someone else on testnet — your community contact, or one of the test merchant accounts — and ask for their vault address. In the VFIDE app, tap *Send*, paste their address, enter an amount (try 1 VFIDE to start), and tap *Send*. Your wallet will ask you to approve. Approve it. Within a minute, the recipient sees the VFIDE arrive in their vault. If the amount is at most 10 VFIDE, the fee is capped at 1 percent (0.01 VFIDE on a 1 VFIDE transfer). For larger amounts, the fee depends on your ProofScore — when you are new, that is around 3.82 percent.

If you remember nothing else

After step 6 you are a fully working VFIDE user. The protections you set up — guardians, vault, signed transfers — are exactly the same protections everyone else has. The system does not have separate tiers for beginners and experts. You are an equal participant from the first transaction.

Power-user quickstart

If you are an auditor, integrator, or developer, here is the operating model in one page.

Repository layout

Top-level under `contracts/`:

- **`contracts/*.sol`** (41 files) — production contracts deployed in Phase 1–2.
- **`contracts/future/*.sol`** (20 files) — Phase 3–6 contracts, deploy when their feature is needed.
- **`contracts/interfaces/*.sol`** (27 files) — pure interface contracts.
- **`contracts/mocks/*.sol`** (11 files) — verifier mocks for testing only.
- **`contracts/pools/*.sol`** (3 files) — DAOPayroll, MerchantCompetition, HeadhunterCompetition pools.
- **`contracts/lib/`, `contracts/libraries/`** — shared libraries (ScoringConstants, Uniswap V3 math).
- **`contracts/legacy/`, `contracts/testnet/`** — legacy stub and testnet faucet.

Five contracts to read first

If you are auditing or integrating and have time for only five files, read these in this order:

#	File	Why
1	<code>contracts/VFIDEToken.sol</code>	ERC-20 token + transfer fee path + anti-whale + vault routing. The hot path on every transfer.
2	<code>contracts/CardBoundVault.sol</code>	User vault. EIP-712 TransferIntent struct, walletEpoch, withdrawal queue, guardian system.
3	<code>contracts/Seer.sol</code>	ProofScore engine. Composition, decay, rate limits, score history, dispute resolution.
4	<code>contracts/ProofScoreBurnRouter.sol</code>	Fee computation. Linear interpolation curve, micro-tx ceiling, sustainability redirects, volume-adaptive multiplier.
5	<code>contracts/DAO.sol</code>	Governance. Proposal types, voting period, fatigue, score-snapshot rules, queue/timelock pipeline.

Critical thresholds at a glance

Constant	Value	Source
MAX_SUPPLY	200,000,000 × 10 ¹ ■	VFIDEToken.sol:52
DEV_RESERVE_SUPPLY	50,000,000 × 10 ¹ ■	VFIDEToken.sol:53
minTotalBps / maxTotalBps	25 / 500 (0.25% — 5%)	ProofScoreBurnRouter.sol:94-95
microTxFeeCeilingBps	100 (1%)	ProofScoreBurnRouter.sol:96
microTxMaxAmount	10 × 10 ¹ ■ (10 VFIDE)	ProofScoreBurnRouter.sol:97
dailyBurnCap	500,000 × 10 ¹ ■	ProofScoreBurnRouter.sol:114
minimumSupplyFloor	50,000,000 × 10 ¹ ■	ProofScoreBurnRouter.sol:118
NEUTRAL / MIN_SCORE / MAX_SCORE	5,000 / 10 / 10,000	Seer.sol:230-232
maxSingleReward	100 (1% per call)	Seer.sol:156
maxDAOScoreChange	500 (5% per call, post H-5 fix)	Seer.sol:172
DAO_SCORE_COOLDOWN	4 hours	Seer.sol:177
MIN_GOVERNANCE / MIN_MERCHANT	5,400 / 5,600	ScoringConstants.sol:32-35
minScoreToEndorse	7,000	SeerSocial.sol:117
minScoreToMentor	7,200	SeerSocial.sol:127
COUNCIL_MIN_SCORE	7,000 (immutable)	CouncilManager.sol:56
councilSize (default)	12 (configurable 1–21)	CouncilElection.sol:97
FIXED_TERM_SECONDS	365 days (immutable)	CouncilElection.sol:25
FIXED_MAX_CONSECUTIVE_TERMS	1 (immutable)	CouncilElection.sol:24
MAX_GUARDIANS	20	CardBoundVault.sol:51
WITHDRAWAL_DELAY	7 days	CardBoundVault.sol:95
SENSITIVE_ADMIN_DELAY	7 days	CardBoundVault.sol:50
MIN_ROTATION_DELAY / MAX_ROTATION_DELAY	10 minutes / 7 days	CardBoundVault.sol:48-49
GUARDIAN_VOTE_WINDOW	14 days	VaultRecoveryClaim.sol:58
CHALLENGE_PERIOD	7 days	VaultRecoveryClaim.sol:56

Constant	Value	Source
ACTIVE_VAULT_CHALLENGE_PERIOD	14 days	VaultRecoveryClaim.sol:138
ESCROW_DURATION (FraudRegistry)	30 days	FraudRegistry.sol:66
HIGH_VALUE_THRESHOLD (Escrow)	10,000 × 10 ¹⁸ ■	EscrowManager.sol:89
ARBITER_TIMELOCK	7 days	EscrowManager.sol:92
DAOTimelock delay (default / floor)	48 hours / 24 hours abs	DAOTimelock.sol:43,107
DAO votingPeriod / votingDelay	7 days / 1 day	DAO.sol:86,89
MAX_PROPOSALS / QUEUE_EXPIRY	200 / 30 days	DAO.sol:123-125
EMERGENCY_RESCUE_DELAY	14 days	DAO.sol:97
FATIGUE_PER_VOTE / RECOVERY_RATE	5% / 5% per day	DAO.sol:163-164
MIN_BURN_BPS / MAX_SINGLE_BPS (FeeDistributor)	2,000 / 5,000 (immutable)	FeeDistributor.sol:37-38
FeeDistributor split default	35 / 20 / 15 / 20 / 10 (burn/sanctum/payroll/merchant/headhunter)	FeeDistributor.sol:167-171
BRIDGE_REFUND_DELAY	7 days	VFIDEBridge.sol:207
bridgeFee (default)	10 bps (0.1%)	VFIDEBridge.sol:62

Deployment in one paragraph

Run `npx hardhat run scripts/deploy-all.ts --network baseSepolia`. This deploys 18 Phase-1 contracts in dependency order with checkpointing to `.deployments/${network}.json` after each successful step. Wait 48 hours for module-setter timelocks to elapse, then run `npx hardhat run scripts/apply-all.ts --network baseSepolia` to wire references. Verify on Basescan. Fund VFIDETestnetFaucet with operator-key VFIDE plus a small ETH budget. Begin.

Mini glossary — 10 terms to know

These are the ten terms that come up most often in the rest of the manual. The full glossary is at the back; this is enough to get started.

VFIDE. The token used in this system, and the system itself. 200 million tokens minted at genesis, no inflation, no future minting.

Vault. Your personal smart-contract account. Tokens live here. Your wallet app is the key, the vault is the bank account.

Wallet. The app on your phone that signs transactions. Holds the private key, never holds tokens directly. MetaMask, Rainbow, Coinbase Wallet, etc.

ProofScore. Your trust number, 0 to 10,000. Starts at 5,000. Goes up with honest behavior, down with disputes. Determines the fees you pay.

Burn fee. The small percentage of every transfer that funds the network. Paid by the sender, not the merchant. From 5% (low trust) down to 0.25% (high trust).

Guardian. A trusted person whose own VFIDE address is listed as someone allowed to help recover your vault if you lose your phone. Two or more required.

Sanctum Fund. A community charity fund that automatically receives 10% of every transfer fee. Disbursements to approved charities require multi-approver review.

Council. The 12-member elected governance body. Members serve fixed one-year terms, must maintain ProofScore $\geq 7,000$, cannot serve consecutive terms.

Endorsement. A signal from a high-trust user (score $\geq 7,000$) that adds a time-limited bonus to another user's score. Builds the social trust graph.

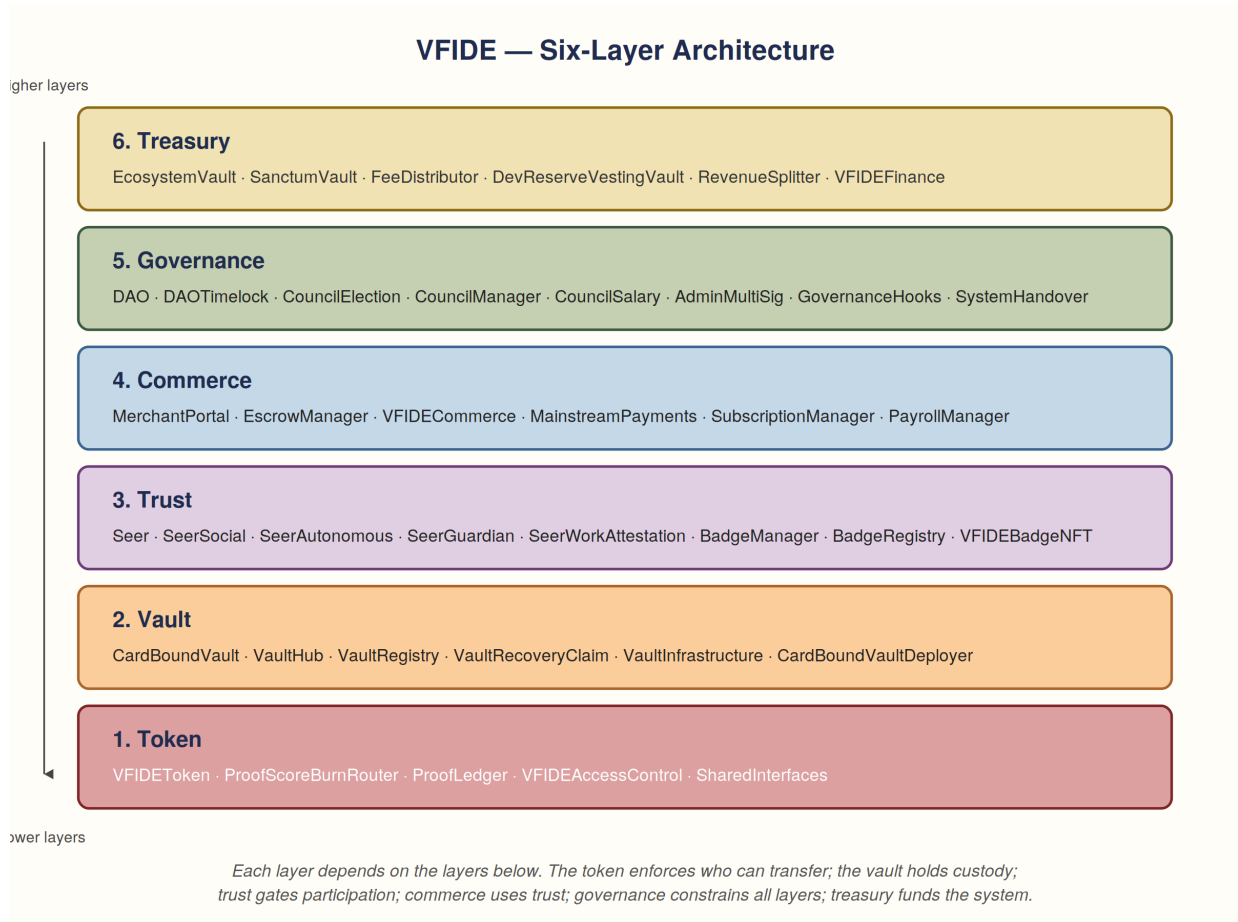
Timelock. A built-in delay between when an administrative change is proposed and when it can take effect. Most are 48 hours. Gives the community time to respond.

Part II

User Manual

Before we start — the big picture

Before diving into the chapters, here is the overall shape of VFIDE. Each chapter that follows zooms in on one of these layers.



The six-layer architecture. Each layer depends on the layers below. Token at the bottom, treasury at the top.

Chapter 1 · Wallets, Vaults, and You

PLAIN ENGLISH

1.1 The wallet is the key. The vault is the bank account.

If you have used a bank, you know the difference between your debit card and your bank account. The card identifies you and authorizes spending; the account is where the money actually sits. If you lose your card, the bank gives you a new one and the money in the account is unchanged.

VFIDE works the same way. The app on your phone — MetaMask, Rainbow, Coinbase Wallet — is a card. The card has a private key inside it that proves you are you and authorizes payments. But the tokens themselves do not live in the card. They live in a smart contract called a *vault*, deployed on the blockchain just for you.

The vault has its own address, separate from your wallet's address. When someone sends you VFIDE, they send it to your vault, not to your wallet. When you spend VFIDE, your wallet signs a message that authorizes the vault to release tokens. The vault checks the message and, if everything is in order, sends the tokens. The wallet never holds the tokens itself, even briefly.

This is why losing your phone does not mean losing your money. The phone had the card; the card is replaceable. The money is in the vault, which lives on the blockchain. To get back into your vault, your guardians (the trusted people you chose during setup) can authorize a new key to take over. Chapter 4 explains exactly how that works.

If you remember nothing else

Your money is not in your phone. It is in a smart contract — your vault — on the blockchain. Your phone is just the key to the vault. Lose the phone, the money is still safe; the guardians help you make a new key.

HOW IT WORKS

1.2 How it works

There are three pieces of infrastructure to know: **VaultHub**, the factory that creates and tracks all vaults; **CardBoundVault**, the per-user contract that holds tokens; and **VaultRegistry**, an optional lookup service that lets you find your vault if you have lost your wallet but remember some recovery information (a hashed phone number, for instance).

How a vault is created

Vaults are deterministic. Before any vault contract is ever deployed, the system can compute what the address of your future vault will be — using a CREATE2 hash of your wallet address and a fixed salt. This means anyone can send VFIDE to your future vault address and the tokens will land safely there, even before the vault contract exists. When the first transfer arrives, the VaultHub deploys the contract atomically inside the same transaction, so the tokens arrive at a real, working contract. The user does not need to do anything in advance.

Why funds live in a contract, not a private key

A regular crypto wallet is a single private key. If someone copies your private key, they can move every token in it instantly. There is no recovery, no second factor, no time delay.

A vault is a contract with rules. Rules like: "transfers above 1,000 VFIDE wait seven days before they execute, and during those seven days my guardians can cancel them." Rules like: "a transfer requires a signed message that includes the current wallet epoch, and the moment I rotate to a new wallet, the epoch increments and every old signature stops working." These rules are enforced by code that no third party can bypass.

A thief who copies your private key gets the wallet, but the wallet is constrained by the vault. The thief can spend up to your daily limit but cannot drain everything. Meanwhile, you and your guardians have time to rotate to a new wallet, which immediately invalidates the thief's signatures. The combination — wallet authorizes, vault enforces — is what makes the system non-custodial without being fragile.

The withdrawal queue

The vault has a setting called the *large transfer threshold*. Transfers below the threshold execute immediately. Transfers at or above the threshold are queued for seven days. During the wait, any one of your guardians can cancel the queued transfer with a single transaction. After seven days, the admin (you, normally) can execute it. This is the strongest protection against a stolen key — even if a thief drains your phone, the largest amount they can move quickly is bounded.

TECHNICAL REFERENCE

1.3 Technical reference

Contracts involved

- **VaultHub** (contracts/VaultHub.sol, 638 LOC) — factory and registry. `ensureVault(owner)` deploys the vault if needed and returns its address. Maintains `vaultOf[owner]` and `ownerOfVault[vault]` mappings.
- **CardBoundVault** (contracts/CardBoundVault.sol, 1,536 LOC) — the user vault. Holds tokens, validates EIP-712 signatures, enforces spend limits, manages guardians and the withdrawal queue.
- **VaultRegistry** (contracts/VaultRegistry.sol, 903 LOC) — optional recovery lookup by hashed identifier.

- **VaultRecoveryClaim** (contracts/VaultRecoveryClaim.sol, 760 LOC) — the "I lost my wallet" recovery flow with a 14-day guardian voting window plus a 7-day challenge period.
- **VaultInfrastructure** (contracts/VaultInfrastructure.sol, 1,333 LOC) — legacy infrastructure and the place where force-recovery functions are deliberately disabled.
- **CardBoundVaultDeployer** (contracts/CardBoundVaultDeployer.sol, 147 LOC) — auxiliary deployment helper.

EIP-712 transfer intents

Every vault-to-vault transfer requires a signed `TransferIntent` from the current active wallet. The struct has seven fields:

```
struct TransferIntent {
    address vault; // source vault
    address toVault; // destination vault
    uint256 amount;
    uint256 nonce; // sequential, prevents replay
    uint64 walletEpoch; // increments on rotation
    uint64 deadline; // expiry timestamp
    uint256 chainId; // prevents cross-chain replay
}
```

The signature uses `secp256k1` with malleability protection (s-value upper bound check). Validation rejects: a deadline in the past, a `chainId` mismatch, a wrong `walletEpoch`, or a non-monotonic nonce. Failure paths revert with named errors: `CBV_InvalidChain`, `CBV_InvalidEpoch`, `CBV_Expired`, `CBV_BadSignature`.

Code citations

Definitions: `TransferIntent` struct `CardBoundVault.sol:203-211`; `TYPEHASH` `CardBoundVault.sol:37-39`. Validation in `executeVaultToVaultTransfer` at `CardBoundVault.sol:774-810`.

Spend limits and the withdrawal queue

- **maxPerTransfer** and **dailyTransferLimit** are configurable per vault, set at construction by `VaultHub` and updatable via `timelocked setSpendLimits()`.
- **WITHDRAWAL_DELAY** = 7 days (`CardBoundVault.sol:95`). Transfers at or above `largeTransferThreshold` queue for this period.
- Maximum 20 active queued withdrawals per vault.
- During the queue period, any guardian may call `cancelQueuedWithdrawal()`; after the delay, admin may execute.

Guardians

- **MAX_GUARDIANS** = 20 (CardBoundVault.sol:51).
- Minimum 2 guardians and threshold ≥ 2 required for guardian setup to be marked complete (and for sensitive operations to be enabled).
- Guardian changes go through **SENSITIVE_ADMIN_DELAY** = 7 days via **proposeGuardianChange** / **applyGuardianChange** (CardBoundVault.sol:50, 537).

Wallet rotation

Rotation is the safe way to change the active wallet (for instance, after a phone replacement, or as a precaution if you suspect a key compromise).

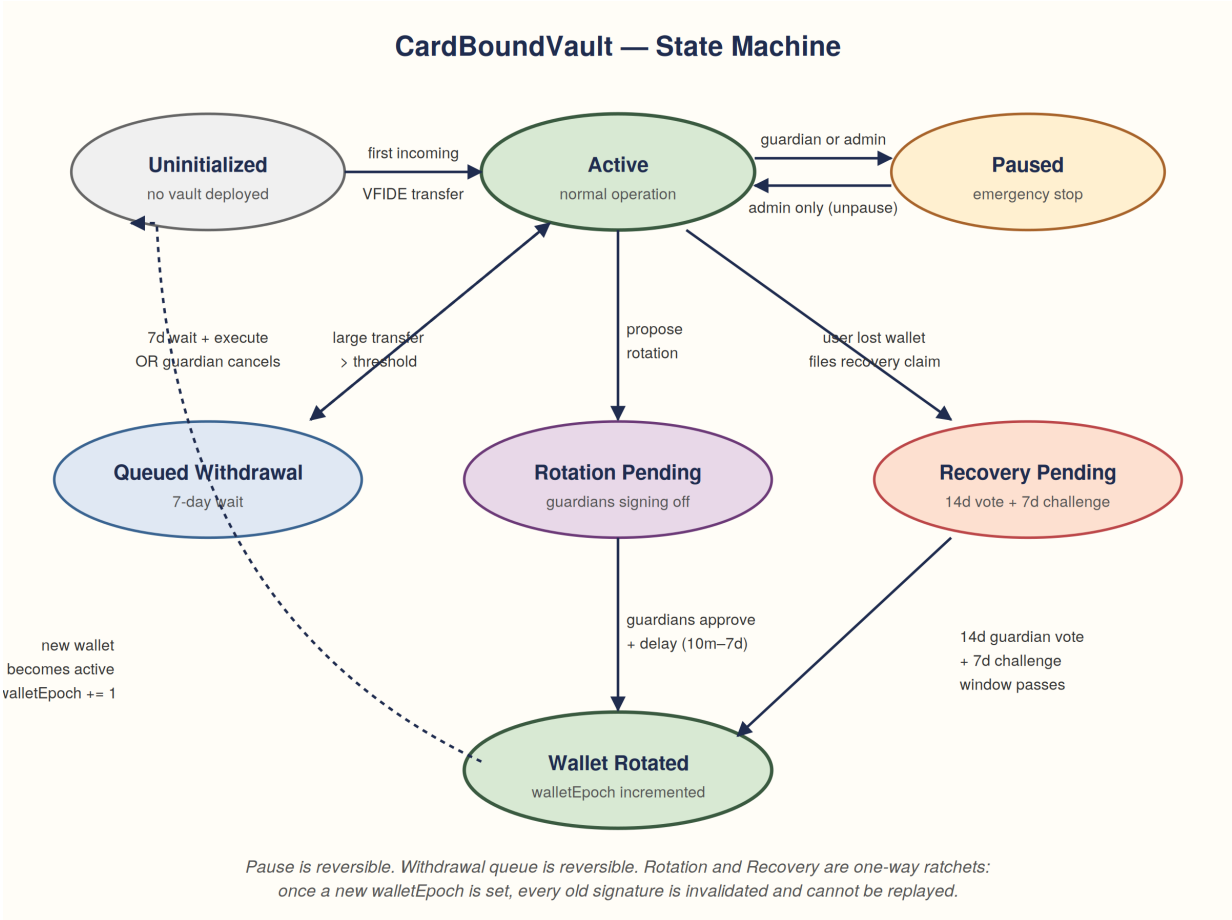
- **proposeWalletRotation(newWallet, delaySeconds)** — admin only, delaySeconds in [MIN_ROTATION_DELAY, MAX_ROTATION_DELAY] = [10 minutes, 7 days].
- Guardians call **approveWalletRotation()** until the threshold is met.
- After the delay AND threshold approvals, anyone may call **finalizeWalletRotation()**.
- On finalize: **activeWallet** changes, **walletEpoch** += 1, every previously signed **TransferIntent** becomes invalid (CardBoundVault.sol:759-762). The nonce does **not** reset.

What is architecturally impossible

These functions exist as stubs that always revert, documenting the deliberate non-custodial design:

- **VaultHub.approveForceRecovery** / **initiateForceRecovery** / **finalizeForceRecovery** — all revert with **VH_RecoveryDisabled** (VaultHub.sol:393-405).
- **VaultInfrastructure** equivalents — all revert with "VI: force recovery disabled - non-custodial" (VaultInfrastructure.sol:1286-1305).
- Recovery is *only* through (a) wallet rotation requiring guardian approval, or (b) **VaultRecoveryClaim**'s 14-day vote + 7-day challenge process — both rely on the user's own chosen guardians.

Vault state machine



The vault has six states. Pause and queued withdrawal are reversible. Rotation and recovery increment walletEpoch and are one-way.

Chapter 2 · Sending and Receiving Payments

PLAIN ENGLISH

2.1 How a payment works

To send VFIDE to someone, you need three things: their vault address, the amount you want to send, and a small amount of network gas (less than one cent on Base, Polygon, or zkSync Era). You enter the address and amount in the VFIDE app, and tap *Send*. Your wallet pops up a confirmation. You approve. The transaction goes to the blockchain, and within seconds the recipient sees the VFIDE land in their vault.

Every transfer has a small fee on top of the amount. The fee is paid by the sender — the merchant or the friend on the other end always receives the full amount you said you would send. The fee is taken automatically by the network and split among three things: a portion is permanently destroyed (this keeps the supply from growing); a portion goes to the Sanctum Fund (the community charity fund); and the rest goes to the ecosystem (operations, council salaries, merchant rewards, headhunter rewards).

How big is the fee? It depends on your ProofScore. A brand-new user pays the most — roughly 3.82 percent on a typical transaction. A trusted user with a high ProofScore pays the least — 0.25 percent. The whole system is built so that as you use it honestly, the cost of using it goes down.

There is one important exception. If you are sending a small amount — 10 VFIDE or less — the fee is capped at 1 percent regardless of your score. This protects new users from paying nearly four percent on small daily purchases like food, transport, or change. If your earned rate is already below 1 percent (because your ProofScore is high), you pay your earned rate, not the cap. Trust is never penalized.

If you remember nothing else

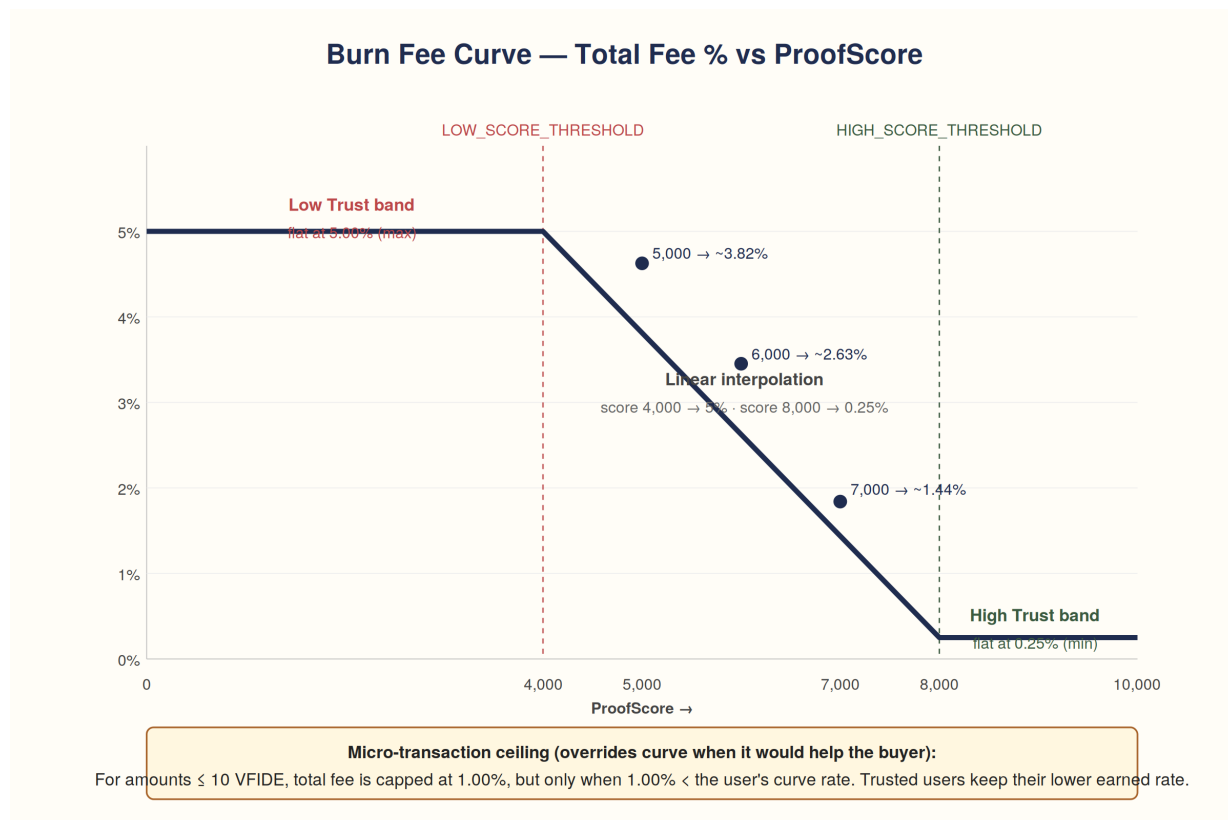
The merchant always receives the full price. You — the sender — pay the small network fee. On amounts of 10 VFIDE or less, the fee is capped at 1 percent. On larger amounts, your ProofScore determines the rate.

HOW IT WORKS

2.2 How it works

The fee curve

The fee is a continuous function of the sender's ProofScore. From score 0 up to 4,000, the fee is flat at the maximum (5 percent). From 4,000 up to 8,000, the fee interpolates linearly down to the minimum (0.25 percent). From 8,000 up to 10,000, the fee is flat at the minimum. The shape:



Burn-fee curve: 5% flat below score 4,000; linear interpolation 4,000–8,000; 0.25% flat above 8,000. Micro-tx ceiling overlays at 1% for transfers ≤ 10 VFIDE.

The score used for the fee is not the user's *current* score, but a 7-day time-weighted average. This prevents a user from manipulating their score upward right before a large transfer — the average smooths out short-term boosts.

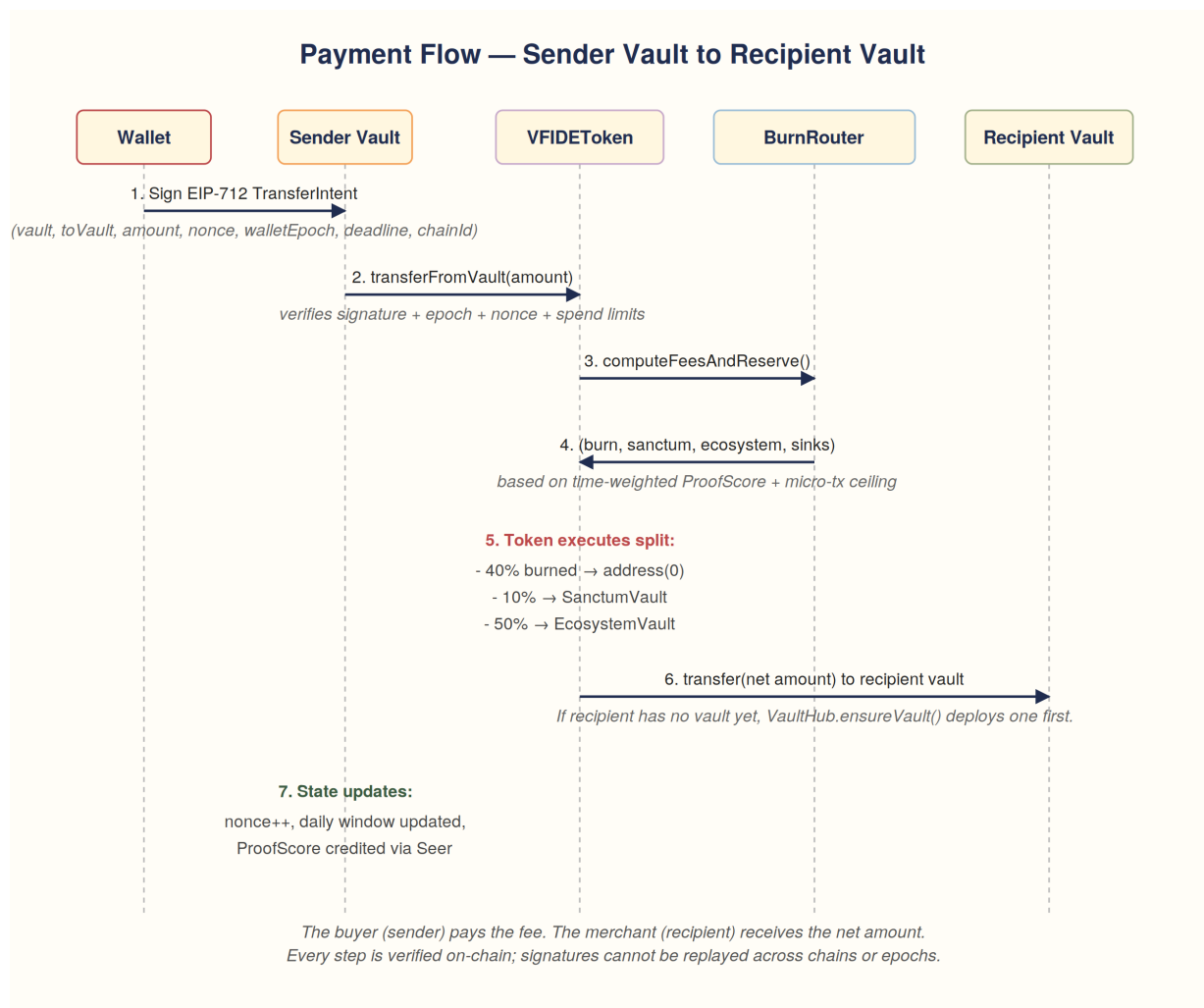
How the fee is split

Whatever the total fee is — 5 percent for a low-trust user, 0.25 percent for a trusted user, 1 percent for a micro-transaction — it is always split the same way at the moment of transfer:

- **40 percent burned.** Permanently removed from circulation. Goes to the burn sink (typically `address(0)`).
- **10 percent to Sanctum.** The community charity fund.
- **50 percent to Ecosystem.** Sent to EcosystemVault, then redistributed by FeeDistributor across five channels (see chapter on Treasury).

Even at the high-trust minimum total fee, an immutable floor of 0.05 percent always reaches the ecosystem fund. This guarantees that operations remain funded regardless of how much trust the user base accumulates.

The full payment flow



Sequence of a vault-to-vault payment. The wallet signs an intent; the vault validates and instructs VFIDEToken; the token consults BurnRouter for the fee split; the token executes the burn, the Sanctum transfer, the ecosystem transfer, and the net delivery to the recipient vault.

Anti-whale limits

VFIDEToken enforces four configurable limits at the contract level. These exist to prevent any single account from market-moving the system on a single transaction. Each can be disabled by the DAO setting it to zero. Exchanges, liquidity pools, and protocol contracts are exempt:

Limit	Default	Purpose
maxTransferAmount	2,000,000 VFIDE (1% of supply)	Caps any single transfer.
maxWalletBalance	4,000,000 VFIDE (2% of supply)	Caps any single wallet/vault balance.
dailyTransferLimit	5,000,000 VFIDE (2.5% of supply)	Rolling 24-hour outflow per sender.
transferCooldown	0 seconds (disabled)	Minimum time between transfers (configurable up).

2.3 Technical reference

Contracts and key paths

- **VFIDEToken** (1,454 LOC) — ERC-20 with custom `_transfer` path. Reads the BurnRouter result, validates returned sink addresses against its own configuration (F-17/C-01 fix), executes the split.
- **ProofScoreBurnRouter** (1,011 LOC) — pure fee computation engine. Stateless from a logic perspective, but maintains daily volume tracking for adaptive multipliers.
- **Seer** — provides the time-weighted ProofScore via `getTimeWeightedScore(address)`.
- **EcosystemVault, SanctumVault** — receivers.

Fee computation step by step

Inside `computeFeesAndReserve(from, to, amount)`:

- Get the time-weighted score: `seer.getTimeWeightedScore(from)`.
- `_calculateLinearFee(score)`: returns `maxTotalBps` if `score ≤ LOW_SCORE_THRESHOLD`; `minTotalBps` if `score ≥ HIGH_SCORE_THRESHOLD`; linear interpolation otherwise (ProofScoreBurnRouter.sol:494-512).
- Apply micro-transaction cap: if `amount ≤ microTxMaxAmount` or `amount-in-USD ≤ microTxMaxUsd6` (when oracle is configured), and `microTxFeeCeilingBps < computed totalBps`, `totalBps` becomes `microTxFeeCeilingBps` (ProofScoreBurnRouter.sol:670-680).
- Split: `burnAmount = totalFee × 40 / 100`; `sanctumAmount = totalFee × 10 / 100`; `ecosystemAmount = totalFee – burn – sanctum` (ProofScoreBurnRouter.sol:684-686).
- Sustainability checks: if supply approaching `minimumSupplyFloor`, burn amounts redirect to ecosystem; if `dailyBurnCap` reached, excess burn redirects to ecosystem (lines 690-758).
- Return six values: (`burnAmount`, `sanctumAmount`, `ecosystemAmount`, `sanctumSink`, `ecosystemSink`, `burnSink`).

VFIDEToken validates the returned sinks

The token does not blindly trust BurnRouter's returned sink addresses. It re-validates each one against its own configured `treasurySink`, `sanctumSink`, and `burnSink`. This was an audit finding (F-17/C-01) — without this validation, a compromised BurnRouter could redirect fees. With it, the worst the BurnRouter can do is revert.

Time-weighted score

The score history is a 50-entry circular buffer per user (Seer.sol:190, `MAX_HISTORY_PER_USER = 50`). `getTimeWeightedScore` walks this buffer and computes a weighted average across the last 7 days. Recent entries weigh more than older ones. This is what protects against a user temporarily inflating their score for a single transfer.

Volume-adaptive fees

When `adaptiveFeesEnabled` is true: below `lowVolumeThreshold` (100,000 VFIDE/day), the multiplier is 1.2x; above `highVolumeThreshold` (5,000,000 VFIDE/day), it is 0.8x; between thresholds, 1.0x. Multiplier defaults disabled; when enabled, applied after the linear curve and the micro-tx cap. `ProofScoreBurnRouter.sol:135-160`.

Sustainability constants

- **dailyBurnCap** = $500,000 \times 10^1$ (0.25% of supply per day).
- **minimumSupplyFloor** = $50,000,000 \times 10^1$ (25% of genesis).
- **ecosystemMinBps** = 5 (0.05%) — the immutable floor for ecosystem allocation.

Errors and revert paths

- **VF_CAP** — would mint above `MAX_SUPPLY` (cannot occur via transfer; defense in depth).
- **VF_DailyExceeded** — sender would exceed `dailyTransferLimit`.
- **VF_MaxTransfer** — single transfer above `maxTransferAmount`.
- **VF_MaxWallet** — recipient would exceed `maxWalletBalance`.
- **VF_Cooldown** — sender within `transferCooldown` window.
- **VF_SinkMismatch** — `BurnRouter` returned an unexpected sink address.

Code citations

Fee curve: `ProofScoreBurnRouter.sol:494-512`. Split: 684-686. Anti-whale: `VFIDEToken.sol:59-62, 1255-1290`. Sink validation: `VFIDEToken.sol:F-17/C-01` fix block. ProofScore time-weighted average: `Seer.sol` `getTimeWeightedScore`.

Chapter 3 · ProofScore — Your Trust Number

PLAIN ENGLISH

3.1 Your reputation, made portable

ProofScore is a single number — between 0 and 10,000 — that represents how trustworthy your behavior on VFIDE has been. Every new user starts at 5,000, the neutral middle. As you use the system honestly, your score rises. As your score rises, you pay lower fees, qualify for better lending terms, gain the right to vote in governance, and gain the right to endorse other users.

ProofScore is not a credit score. A credit score is calculated by a private company you have no relationship with, based on data they bought from other companies. ProofScore is calculated by code that anyone can inspect, based on actions you took. There is no inquiry. There is no permanent record of a mistake you made ten years ago. If you behave honestly, your score recovers. If you stop using the system, your score drifts back toward neutral.

ProofScore is portable. It belongs to you and travels with you. The trust you build selling fruit at one market is valid at every other market, with every lender who runs on VFIDE, in every governance vote you cast. You cannot buy it. You cannot transfer it. You cannot inherit it. You earn it by what you do.

Most importantly: the score has rights attached. You have the right to know why your score changed (every change records a reason). You have the right to challenge a score reduction you believe was unfair (the dispute system handles this). You have the right to bounded penalties — no single action can cost you more than a small fixed percentage. And you have the right to recovery — penalties decay over time, and continued good behavior restores your score.

If you remember nothing else

ProofScore is your trust number. It starts at 5,000. It goes up with honest behavior, down with disputes, and slowly toward 5,000 if you stop using the system. Higher score = lower fees and more privileges. The score belongs to you.

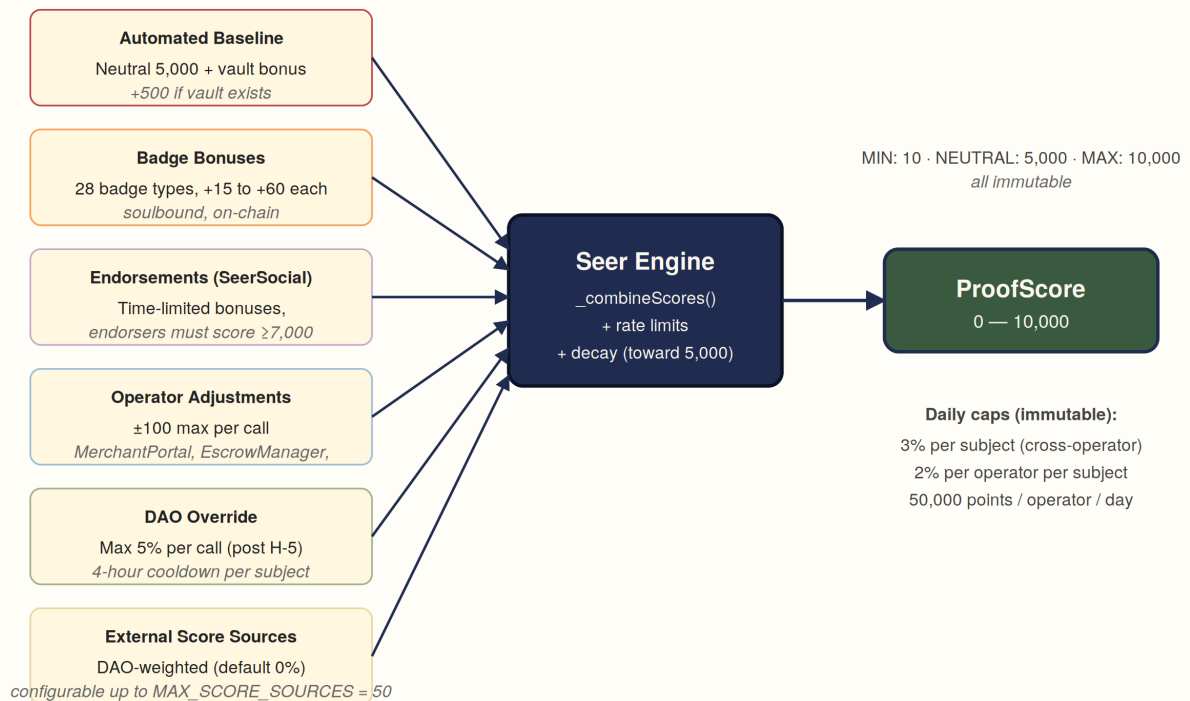
HOW IT WORKS

3.2 How it works

Where the score comes from

ProofScore is computed by the Seer contract, which combines several inputs:

ProofScore — How It Is Composed



ProofScore composition. Six contributing inputs are combined by Seer's engine, subject to per-call and per-day rate limits, and bounded between MIN_SCORE and MAX_SCORE.

In plain terms:

- **The automated baseline** starts every user at 5,000, plus 500 if they have a vault, plus bonuses from any earned badges and any active endorsements from trusted peers.
- **Operator adjustments** are small score changes (rewards or penalties) applied by authorized contracts when you take specific actions: completing a successful transaction, fulfilling an escrow, defaulting on a loan, etc. Each adjustment is small (max 1 percent per call), and the daily total is capped.
- **DAO overrides** are larger score changes the DAO can apply for migrations, dispute resolutions, or rectifications. After audit fix H-5, these are limited to 5 percent per call with a 4-hour cooldown per subject.
- **External score sources** are an extension mechanism: the DAO can register external contracts that contribute to a user's score with configurable weights. By default these have zero weight; over time the community can shift weight from DAO-set scores to objective on-chain measures.

Score thresholds and what they unlock

Score range	What it does
0 – 4,000	Low Trust band. Fee is flat 5%. Cannot register as merchant, cannot vote.
4,000 – 5,400	Improving. Fee curves down from 5% toward 2.34%. Can borrow.
5,000	Neutral. Default starting point for new users.
5,400+	Governance Eligible. Can vote in DAO proposals; can be a council candidate.
5,600+	Merchant Eligible. Can register as a merchant in MerchantPortal.
6,000+	Endorser-curve threshold. Above this point the curve drops below ~2.6%.
7,000+	Endorsement Eligible (default). Can endorse others. Council eligibility minimum.
7,200+	Mentorship Eligible (default). Can formally sponsor newcomers in the social layer.
8,000 – 10,000	High Trust band. Fee is flat 0.25%. 3-day escrow as merchant. Can file fraud complaints. Premium features.

Score decay

If you stop using the system, your score drifts toward 5,000. Decay begins after a configurable period of inactivity (default 90 days) and removes a small amount per month (default 100 points). High scores decay downward; low scores decay upward. Both converge on neutral over time. This means trust must be actively maintained — you cannot earn a perfect score, disappear for a year, and return with a perfect score.

Rate limits prevent manipulation

Without limits, a single operator could push a user's score up or down quickly. Seer enforces caps at every level:

- **Per-call:** 100 points (1%) maximum on any single reward() or punish() call.
- **Per-operator per-subject per day:** up to 200 points by default.
- **Per-operator globally per day:** 50,000 points across all subjects.
- **Cross-operator per subject per day:** 300 points (3%) total from all operators combined — the master cap.
- **DAO setScore:** 500 points (5%) max per call, 4-hour cooldown per subject.

These bounds are immutable for the score range itself (0–10,000), the minimum (10), and the cross-operator daily cap (3%). The other values are configurable by the DAO with timelock, but only within hardcoded ceilings.

TECHNICAL REFERENCE

3.3 Technical reference

Contracts

- **Seer** (1,403 LOC) — primary score engine. Score storage, composition, dispute resolution, score history.
- **SeerSocial** (736 LOC) — endorsements and mentorship system.
- **SeerAutonomous** (1,320 LOC) — automated pattern detection (velocity anomalies, structuring detection).
- **SeerGuardian** (648 LOC) — mutual oversight between DAO and Seer.
- **SeerWorkAttestation** (436 LOC) — on-chain proof of work events that feed score.
- **SeerView** (232 LOC) — read-only view extension.
- **SeerPolicyGuard** (213 LOC) — policy version pinning.
- **SeerAutonomousLib** (84 LOC) — shared internal logic.
- **ScoringConstants** (contracts/lib/ScoringConstants.sol) — canonical thresholds shared by all consumers.

Constants in Seer

Constant	Value	Notes
MIN_SCORE	10	Hard floor (Seer.sol:230).
MAX_SCORE	10000	Hard ceiling (Seer.sol:231).
NEUTRAL	5000	New-user default (Seer.sol:232).
MAX_SCORE_SOURCES	50	Cap on external score-source registrations (Seer.sol:407, I-11 fix).
maxSingleReward	100	Configurable up to 500 (Seer.sol:156, 854).
maxOperatorDailyReward	200 (default)	Per-operator per-subject daily cap.
maxDailyOperatorGlobalReward	50000	Per-operator daily cap across all subjects (Seer.sol:159).
maxDailySubjectDelta	300 (3% — immutable cap)	Cross-operator per-subject daily cap (Seer.sol:169).
maxDAOScoreChange	500 (5%)	Reduced from 2000 in H-5 fix (Seer.sol:172).
DAO_SCORE_COOLDOWN	4 hours	Increased from 1 hour in H-5 fix (Seer.sol:177).
MAX_HISTORY_PER_USER	50	Score history circular buffer (Seer.sol:190-194).

Key functions

- `getScore(address)` returns (uint16) — current effective score; uses cached value when external sources are weighted, otherwise computes.
- `getCachedScore(address)` returns (uint16) — gas-bounded read.
- `getTimeWeightedScore(address)` returns (uint16) — 7-day weighted average from score history; used by BurnRouter.
- `refreshScoreCache(address)` — anyone may call to update the cache (paying gas).
- `reward(subject, delta, reason)` / `punish(subject, delta, reason)` — operator-only score adjustments.
- `setScore(subject, newScore, reason)` — DAO override.
- `requestScoreReview(reason)` / `resolveScoreDispute(subject, approved, adjustment)` — dispute path.

Score thresholds (canonical, from `ScoringConstants.sol`)

Constant	Value	Used by
LOW_FEE_FLOOR	4,000	BurnRouter — pays max fee at or below.
HIGH_FEE_CEIL	8,000	BurnRouter — pays min fee at or above.
NEUTRAL	5,000	Default starting point.
TIER_1 / TIER_2 / TIER_3 / TIER_4	5,000 / 6,000 / 7,000 / 8,000	VFIDETermLoan tier gating.
MIN_MERCHANT	5,600	MerchantPortal registration.
MIN_GOVERNANCE	5,400	DAO proposal eligibility, vote eligibility.

Score history

Every score change pushes a record into a 50-entry circular buffer per user: `scoreHistory[address][50]`, with `scoreHistoryHead` and `scoreHistoryCount` tracking the write pointer and live entry count (Seer.sol:190-194; S-05 fix). Each entry includes the old score, new score, reason code, source, and timestamp. The buffer is $O(1)$ for writes and $O(n)$ for reads, bounded to 50.

Rights of every user

From the Seer Constitution (encoded in contract behavior, not policy):

- **Right to reason.** Every score change emits an event with a reason code; off-chain UIs can show the user why.
- **Right to challenge.** Disputes can be filed via `requestScoreReview`.

- **Right to bounded penalties.** The cross-operator 3% daily cap is enforced and immutable.
- **Right to equal treatment.** Score adjustments are policy-driven; operators have hardcoded ceilings.
- **Right to recovery.** Penalties decay; `decayEnabled`, when true, pulls all scores toward NEUTRAL over time.

Code citations

Constants: `Seer.sol:156-194, 230-232, 407`. ScoringConstants library: `contracts/lib/ScoringConstants.sol`. Score history circular buffer: `Seer.sol:190-194` (S-05 fix). H-5 DAO change tightening: `Seer.sol:171-177`.

Chapter 4 · Guardians and Recovery

PLAIN ENGLISH

4.1 The people who help when things go wrong

Phones break. Phones get stolen. People forget passwords. In the regular financial world, when something goes wrong with your bank card, you call the bank, prove who you are with documents and security questions, and they cancel the old card and issue a new one. VFIDE has no central authority to call. Instead, you choose your own helpers in advance. They are called *guardians*.

A guardian is a person you trust in real life — your sister, your neighbor, your church leader, the shopkeeper next to your stall. Each guardian has their own VFIDE account. When you set up your vault, you list your guardians' VFIDE addresses. From that point on, your guardians have one specific power: they can authorize a new wallet to take over your vault if you ever lose access to the old one.

Guardians cannot spend your money. They cannot move tokens out of your vault. They cannot freeze your account. The only thing they can do is help you change keys when you have lost access. And they cannot do that alone — it requires a threshold of guardians (you choose how many) acting together.

Pick your guardians carefully. A good guardian is someone who will respond quickly if you ask for help, who keeps their own VFIDE account secure, and who you trust not to collude with anyone trying to steal from you. Three guardians with a threshold of two is a common safe choice — you need any two of them to recover, but no single guardian alone can do anything. Family members in different households work well. Two guardians in the same household is risky if a thief takes both phones at once.

If you remember nothing else

Choose 2 to 3 guardians who are trusted, responsive, and ideally not all in the same place. They cannot spend your money. They can only help you switch to a new key if you lose your phone. Pick now, before you need them.

HOW IT WORKS

4.2 How it works

Two recovery paths

There are two distinct recovery scenarios, each with its own flow:

Path A — Wallet rotation (you still control the old wallet, but want to switch keys)

This is the normal "I got a new phone" flow, or the precautionary "I think my key might be compromised, let me rotate before something happens" flow. From your existing wallet (the admin), you call `proposeWalletRotation(newWalletAddress, delaySeconds)`. The delay can be anywhere from 10 minutes to 7 days — you pick. Then your guardians call `approveWalletRotation()` from their own wallets. Once you have enough guardian approvals to meet the threshold, and the delay you set has elapsed, anyone can call `finalizeWalletRotation()`. The new wallet becomes active. The vault's *walletEpoch* increments by one. Every signature the old wallet ever made becomes invalid.

Path B — Recovery claim (you have lost the old wallet entirely)

If your phone is gone and you have no way to call `proposeWalletRotation`, you use `VaultRecoveryClaim` instead. Find your vault address — either you remember it, or you wrote it down, or you registered a recovery identifier (a hashed phone number, for instance) in `VaultRegistry` beforehand. Then file a recovery claim with: the vault address, your new wallet address, evidence of identity, and an explanation.

The claim then enters a multi-step process:

- **Guardian voting window — 14 days.** Your guardians vote to approve the recovery. At least the threshold must approve.
- **Challenge period — 7 days (or 14 days if the vault has had recent activity).** The original wallet has time to cancel the recovery if it is still in someone's hands. The longer challenge applies if the vault was active in the last 30 days, on the principle that if the vault is being used, the original owner might still be the one using it.
- **Optional identity verification** through trusted oracles, when the deployment configures one.
- **Execution.** If guardians have approved and no challenge has been raised, anyone can execute the claim. The vault's owner becomes the new wallet.

What guardians cannot do

It is worth being explicit about the limits of guardian power, since the system is designed to constrain them as much as it empowers them:

- Guardians cannot spend tokens from the vault. The vault's balance can only be moved by a signed `TransferIntent` from the active wallet.
- Guardians cannot remove other guardians or change the threshold. Those changes go through `SENSITIVE_ADMIN_DELAY` (7 days) and require admin proposing them.
- A single guardian cannot rotate the wallet alone — the threshold (≥ 2) must be met.
- Guardians cannot bypass the propose-finalize delay. Even if all guardians approve instantly, the delay you set still has to elapse.
- Guardians can pause the vault (emergency stop), but only the admin can unpause it. Pause expires automatically after 7 days.

Why the wallet epoch matters

Each vault has a *walletEpoch* counter. Every *TransferIntent* signature is bound to a specific epoch. When a wallet rotation completes, the epoch increments. From that moment forward, every signature made by the old wallet — including any pre-signed transfers a thief might be holding — is invalid. There is no way to replay them.

This is why rotation is a hard reset against compromise. You do not need to know which specific signatures the attacker has. The single act of completing a rotation invalidates everything they could possibly hold.

TECHNICAL REFERENCE

4.3 Technical reference

Contracts

- **CardBoundVault** — guardian list, threshold, propose/apply guardian changes, propose/approve/finalize wallet rotation, pause/unpause.
- **VaultHub** — tracks which vaults have `guardianSetupComplete[vault]`, gates sensitive operations on completion.
- **VaultRegistry** (903 LOC) — optional recovery identifier lookup.
- **VaultRecoveryClaim** (760 LOC) — guardian-voting recovery flow.

Constants

Constant	Value	Source
MAX_GUARDIANS	20	CardBoundVault.sol:51
min guardians for setup completion	2	Implicit in setup-completion check at CardBoundVault.sol:508
min guardian threshold for setup completion	2	Same.
SENSITIVE_ADMIN_DELAY	7 days	CardBoundVault.sol:50
MIN_ROTATION_DELAY	10 minutes	CardBoundVault.sol:48
MAX_ROTATION_DELAY	7 days	CardBoundVault.sol:49
MAX_PAUSE_WINDOW	7 days	CardBoundVault.sol:72
GUARDIAN_MATURITY_PERIOD	7 days	CardBoundVault.sol:82
WITHDRAWAL_DELAY	7 days	CardBoundVault.sol:95
GUARDIAN_VOTE_WINDOW (recovery)	14 days	VaultRecoveryClaim.sol:58
CHALLENGE_PERIOD (recovery)	7 days	VaultRecoveryClaim.sol:56
ACTIVE_VAULT_CHALLENGE_PERIOD	14 days	VaultRecoveryClaim.sol:138
Active-vault detection window	30 days	VaultRecoveryClaim.sol:135 (F-54 fix)

Key functions in CardBoundVault

- `proposeGuardianChange(newGuardians[], newThreshold)` — admin only; queues for SENSITIVE_ADMIN_DELAY.
- `applyGuardianChange()` — anyone may call after delay elapses.
- `proposeWalletRotation(newWallet, delaySeconds)` — admin only; $\text{delay} \in [\text{MIN_ROTATION_DELAY}, \text{MAX_ROTATION_DELAY}]$.
- `approveWalletRotation()` — guardian only; counts toward threshold.
- `finalizeWalletRotation()` — anyone after delay AND threshold met; increments `walletEpoch`.
- `cancelWalletRotation()` — admin or guardian during delay window.
- `pauseVault()` — admin or guardian; auto-expires after MAX_PAUSE_WINDOW (7 days).
- `unpauseVault()` — admin only.

Key functions in VaultRecoveryClaim

- `fileRecoveryClaim(vault, newWallet, identityHash, reason)` — anyone with vault address.

- `guardianApproveRecovery(claimId)` — guardian-only vote within 14-day window.
- `cancelRecoveryClaim(claimId)` — original wallet within challenge period.
- `executeRecoveryClaim(claimId)` — anyone after challenge period if guardian threshold met and no cancel.

Pause semantics

Pause halts vault transfers. The flag is set instantly (no timelock) since it is a defensive action. Both the admin and any guardian may pause. Only the admin can unpause. If the admin is unavailable (lost wallet) and a pause was triggered, the pause expires automatically after `MAX_PAUSE_WINDOW` (7 days). This prevents a malicious guardian from indefinitely freezing a user's vault.

Code citations

Guardian system: `CardBoundVault.sol`:373-425 (constructor), 494-580 (changes & threshold). Wallet rotation: `CardBoundVault.sol`:707-770. Withdrawal queue: `CardBoundVault.sol`:1028-1200. Recovery flow: `VaultRecoveryClaim.sol`:135-345 (active-vault challenge logic from F-54), 425-480 (challenge), 472-540 (execution). Force-recovery disablement (defense by absence): `VaultHub.sol`:393-405; `VaultInfrastructure.sol`:1286-1305.

Chapter 5 · For Merchants

PLAIN ENGLISH

5.1 Why VFIDE inverts the merchant-payment model

Every traditional payment system charges the merchant a cut. Visa and Mastercard take 1.5 to 3.5 percent. PayPal takes 3.49 percent. Stripe takes 2.9 percent plus 30 cents. Square takes 2.6 percent plus 10 cents. The merchant — the person doing the work — pays the cost of using the network. On a 5 percent margin, a 3 percent processing fee eats most of the profit.

VFIDE charges the merchant zero. Not zero-after-discounts, not zero-with-conditions, not zero-for-the-first-six-months. Zero, at the contract level. The setting is called `protocolFeeBps` and it is set to 0 — checked into the code, deployed on the testnet, verifiable on the blockchain by anyone.

The buyer pays a small network fee — exactly the same fee they would pay on any other VFIDE transfer, displayed as a visible line item on the receipt. The merchant receives the full listed price. There is no skimming, no hidden cut, no "processing" that mysteriously consumes 3 percent of every sale.

To register as a merchant on VFIDE, you need a ProofScore of at least 5,600. New users start at 5,000, so the threshold is reachable with a few weeks of regular activity — completing transactions, building endorsements, perhaps mentoring a newcomer. Once registered, you receive payments to your vault, just like any other user. The payment flow is identical to a person-to-person transfer.

If you remember nothing else

VFIDE charges the merchant zero. The buyer pays the network fee, which is small and visible. To register as a merchant, you need a ProofScore of at least 5,600 — reachable with regular honest use of the system.

HOW IT WORKS

5.2 How it works

Registration

MerchantPortal is the contract that tracks merchants. Calling `registerMerchant(metadata)` from your wallet — when your ProofScore is at least 5,600 — adds your address to the merchant registry. Metadata includes business name, category, location (or a hashed equivalent for privacy), and any payment-acceptance details. Registration is free except for transaction gas.

Once registered, you appear in the merchant directory (if you opted in) and can be reached by buyers via the public checkout flow. The portal also tracks aggregate metrics for your business — transaction count, dispute count, refund count — which are used to assign you to a tier (new, established, trusted) that determines your escrow lock period.

Disputes and the auto-suspend mechanism

MerchantPortal monitors your dispute and refund counts. If they exceed configurable thresholds, the contract auto-suspends merchant features pending DAO review. This is not a freeze of your tokens — it only suspends your status as a registered merchant. Tokens in your vault remain yours; you can still send and receive in person-to-person mode. The DAO reviews the underlying disputes and either reinstates the registration or escalates to a permanent removal.

Escrow tiers — your lock period shrinks as you earn trust

When a buyer pays a merchant through the EscrowManager (used for transactions where the buyer wants protection), the funds are locked for a period that depends on the merchant's ProofScore:

Merchant tier	ProofScore range	Escrow lock period
New	below 6,000	14 days
Established	6,000 – 8,000	7 days
Trusted	above 8,000	3 days (immutable minimum)

Compare this to PayPal, which holds new-seller funds for 21 days regardless of the seller's history. VFIDE's adaptive lock period rewards proven track records with faster access to working capital. For a small merchant whose business runs on cash flow, this difference is the difference between being able to buy supplies for the next week and being unable to.

The 13 merchant systems

VFIDE includes a complete merchant operating system at the application layer — the user interface and API routes for everything a small business in an emerging market needs. These are not separate apps to install; they are built into the same VFIDE app the merchant already uses. The thirteen systems:

System	What it does
Invoicing	Create and send invoices with payment links and Smart QR codes; PDF generation.
CRM	Customer database with transaction history, notes, segmentation, lifecycle tracking.
Inventory	Product catalog with variants, stock tracking, low-stock alerts, barcode support.
Refunds	Full or partial refunds with reason codes and audit trail.
Loyalty	Points-based loyalty with configurable earn/burn rates and tier management.
Bookkeeping	Automatic transaction ledger, P&L summaries, exportable.
Tax	Configurable tax rates per jurisdiction with automatic calculation.
Booking	Appointment scheduling with availability management.
Tips	Optional tipping with configurable presets (10%, 15%, 20%, custom).

System	What it does
Discounts	Percentage or fixed discounts with coupon codes, validity periods, usage limits.
Payment Links	Shareable URLs for remote payments; public checkout, no auth required for payer.
Recurring	Subscription billing with plan creation, auto-collection, lifecycle tracking.
Staff Access	Multi-user access with role-based permissions (owner, manager, cashier).

These are not afterthoughts or upsells. They are the reason a fruit vendor in Kumasi does not need to subscribe to Square, Shopify, and QuickBooks. Everything is included. Everything is designed for someone whose primary device is a low-end Android phone with intermittent internet. Many of these systems have offline modes — the POS continues to accept payments without internet and syncs when connectivity returns.

TECHNICAL REFERENCE

5.3 Technical reference

Contracts

- **MerchantPortal** (1,217 LOC) — primary merchant interface. Registration, transaction processing, dispute counts, refund counts, auto-suspend, status queries.
- **EscrowManager** (433 LOC) — high-security escrow for trades requiring arbiter resolution. Tier-based lock periods, DAO-reviewed disputes for amounts > HIGH_VALUE_THRESHOLD.
- **VFIDECommerce** (411 LOC) — checkout and storefront helpers.
- **MainstreamPayments** (1,432 LOC) — POS terminal registry, session-key management, fiat on/off ramp integration points, multi-currency router. Currently in contracts/future/ for Phase 3.
- **SubscriptionManager** (616 LOC) — recurring billing. In contracts/future/.
- **PayrollManager** (761 LOC) — staff payroll for merchants who employ others.

Critical constants

Constant	Value	Source
protocolFeeBps	0 (immutable as deployed)	MerchantPortal.sol:176
minForMerchant (default)	5,600	Seer.sol via ScoringConstants. MIN_MERCHANT
HIGH_VALUE_THRESHOLD	10,000 × 10 ¹⁸ ■	EscrowManager.sol:89
ARBITER_TIMELOCK	7 days	EscrowManager.sol:92
dispute timeout (DAO non-response)	90 days	EscrowManager dispute path
min lock period (Trusted)	3 days (immutable)	EscrowManager

Merchant API surface (28 sub-routes under /api/merchant/)

The frontend exposes a broad merchant API. Each sub-route lives at `app/api/merchant/<name>/route.ts`:

Subroute	Purpose
analytics	Aggregated business metrics (P&L, daily totals).
bookings	Appointment scheduling.
categories	Customer segmentation.
checkout	Payment-link creation and public checkout.
coupons	Discount codes.
customers	CRM.
digital	Digital product delivery.
directory	Public merchant directory listing.
expenses	Business expense tracking for bookkeeping.
gift-cards	Gift card issuance and redemption.
installments	Installment-payment plans.
invoices	Invoice CRUD with PDF generation.
locations	Multi-location support.
loyalty	Loyalty program rules and balances.
orders	Order management.
payments	Payment recording and refunds.
products	Inventory.
profile	Merchant profile and metadata.
receipts	Receipt generation (WhatsApp/SMS-deliverable).
returns	Refund and return processing.
reviews	Customer reviews and merchant responses.
staff	Role-based staff access.
subscriptions	Recurring billing.
suppliers	Supplier management.
training	Built-in onboarding/training content.
webhooks	Outbound webhook delivery for integrations.

Subroute	Purpose
wholesale	B2B wholesale pricing.
withdraw	Off-ramp request flows.

Frontend component locations

- `components/invoice/` — invoice creation and management.
- `components/customers/` — CRM.
- `components/inventory/` — product catalog.
- `components/commerce/` — refunds, tax, tips.
- `components/loyalty/` — loyalty program.
- `components/bookkeeping/` — bookkeeping.
- `components/booking/` — appointments.
- `components/discounts/` — coupons and discounts.
- `components/payment-links/` — payment-link generator.
- `components/recurring/`, `components/social/SubscriptionManager.tsx` — subscription/recurring billing.
- `components/merchant/` — staff access, dashboard, POS, payment interface.
- `components/checkout/` — public checkout panel.
- `components/qr/`, `components/pos/` — Smart QR and offline POS.

Auto-suspend mechanism

MerchantPortal tracks `refundCount[merchant]` and `disputeCount[merchant]`. When either exceeds a configurable threshold (set by DAO), `autoSuspendIfThresholdExceeded()` can be called. The merchant's status flag flips to suspended. Tokens remain in the merchant's vault — only the merchant *role* is affected. The DAO reviews and either restores or escalates.

Code citations

Merchant registration: MerchantPortal.sol:400-450 (registerMerchant, eligibility check at 416). Protocol fee: MerchantPortal.sol:176 (immutable as 0). Escrow tiers: EscrowManager.sol lock-period logic. API surface: `app/api/merchant/*` (28 sub-route directories). Frontend systems: `components/{invoice,customers,inventory,commerce,loyalty,bookkeeping,booking,discounts,payment-links,recurring,social,merchant,checkout,qr,pos}/`.

Chapter 6 · Governance — Voting and the Council

PLAIN ENGLISH

6.1 Voting with your trust, not your money

VFIDE is governed by an elected twelve-member council, not by token holders and not by a founding team. The council sets protocol parameters, approves charity distributions, and decides the high-stakes disputes. After the six-month bootstrap period, the council holds the protocol's administrative authority. The developer's key is burned by then.

What makes VFIDE's governance unusual is that **voting power is your ProofScore, not your token balance**. A speculator who bought a million VFIDE on day one and did nothing else has zero voting power until they earn a ProofScore. A market vendor who has been transacting honestly for two years has more voting power than almost anyone, because her score is high. Trust votes. Money does not.

The council has exactly twelve seats, served by people elected by the community. Each council member serves a one-year fixed term and cannot serve consecutive terms — after their year is up, they step down and someone else takes the seat. This prevents entrenchment. To stay on the council, members must keep their ProofScore at 7,000 or higher; if they drop below, they have a seven-day grace period to recover, then they're automatically removed.

Anyone with a ProofScore of 5,400 or higher can submit proposals to the DAO. Anyone with 5,400 or higher can vote. The proposal goes through a one-day delay (to prevent flash-loan attacks where someone temporarily inflates their voting weight), then voting is open for seven days, then there's a forty-eight-hour timelock before execution. Every step is visible on-chain. There are no surprises.

If you remember nothing else

Trust votes, not money. The council has 12 seats, fixed 1-year terms, no consecutive terms. Anyone with ProofScore $\geq 5,400$ can vote. There is a 48-hour timelock before any approved proposal can execute, so the community always has time to respond.

HOW IT WORKS

6.2 How it works

Proposal types

Every proposal is one of four types, each gated by the function selector and target address it can call. The DAO maintains an explicit allow-list of which (target contract, function selector) pairs each proposal type can invoke.

Type	Use	Examples
Generic	General protocol decisions that don't move tokens or change critical params.	Update a public-display parameter, configure a non-critical hook.
Financial	Anything involving token transfers or treasury allocation.	Sanctum charity disbursements, ecosystem grant payouts, council salary parameter changes.
ProtocolChange	Modifies core protocol parameters — fees, thresholds, score weights.	Adjust LOW/HIGH fee thresholds, change microtx ceiling, set ProofScore source weights.
SecurityAction	Emergency security responses.	Activate circuit breaker, propose Seer policy update, change CouncilSalary recipients.

Each type has a configurable allow-list of target contracts and function selectors (`proposalTypeTargetAllowed`, `proposalTypeSelectorAllowed`). A Generic proposal cannot execute a SecurityAction-tier function, even if the council wanted it to — the structural restriction prevents miscategorization and abuse.

Voting mechanics

When a proposal is created, the DAO records every eligible voter's score at that moment as the snapshot. Vote weight = ProofScore at the snapshot deadline. This freezing prevents anyone from boosting their score during a vote to gain extra weight. The score must also have been **established at least two days before** the proposal was created (`SCORE_SETTLEMENT_WINDOW` = 2 days), which prevents last-minute coordinated score-pumping by an attacker.

Voters who participate frequently accumulate **fatigue**. Each vote adds 5% to a fatigue counter, and fatigue recovers at 5% per day. Voters whose fatigue exceeds zero have their effective vote weight reduced. This prevents a small group of dedicated voters from steamrolling all the proposals; it makes thoughtful, occasional voting more powerful per-vote than spam voting.

Voters must have been active in the last ninety days. A wallet that hasn't moved tokens or signed any meaningful transaction in three months cannot vote — the score may still be there, but it's stale, and stale scores are excluded from live governance to prevent dormant-account voting attacks.

Quorum and execution

For a proposal to pass, three things must happen: (1) more than 50% of the weighted votes must be in favor, (2) the total vote points must reach the `minVotesRequired` threshold (configurable by the DAO, with a `syncQuorumProfile` mechanism that auto-tunes it as the network grows), and (3) at least `minParticipation` distinct voters must participate (default ten unique addresses).

Once a proposal passes, it's queued in the `DAOTimelock` for at least forty-eight hours before execution. During that window anyone can read the proposal data and react. After the timelock, anyone can call `execute`. If execution doesn't happen within thirty days of queuing, the proposal expires and must be re-proposed.

Emergency quorum rescue

Sometimes governance can deadlock — quorum becomes unreachable because the active voter base shrinks below the threshold. To prevent permanent paralysis, the DAO has an emergency rescue path. The admin can initiate a quorum reduction (`EMERGENCY_RESCUE_DELAY = 14` days). The same person cannot self-approve. After both initiator and a second-line approver have signed off and the fourteen-day visibility window has elapsed, the rescue executes and lowers the quorum requirement so governance can move again. The fourteen-day delay exists so the community has two full weeks to mobilize, build a counter-proposal, or override before the rescue takes effect.

The break-glass admin path

There is a separate seven-day delay (`BREAK_GLASS_DELAY = 7` days) for an admin to be replaced without going through the normal timelock. This is for the case where the timelock itself is broken or admin control is otherwise lost. It is intentionally slow.

Council mechanics

Members are elected via the `CouncilElection` contract. Nominations require `ProofScore ≥ 7,000` (the council minimum). Voting is `ProofScore`-weighted, the same as any DAO proposal. The contract's `MAX_COUNCIL_SIZE` is 21, but the active council size defaults to 12 (configurable up to 21 by the DAO as the network grows). `FIXED_TERM_SECONDS = 365` days and `FIXED_MAX_CONSECUTIVE_TERMS = 1` are immutable — they cannot be changed by any vote.

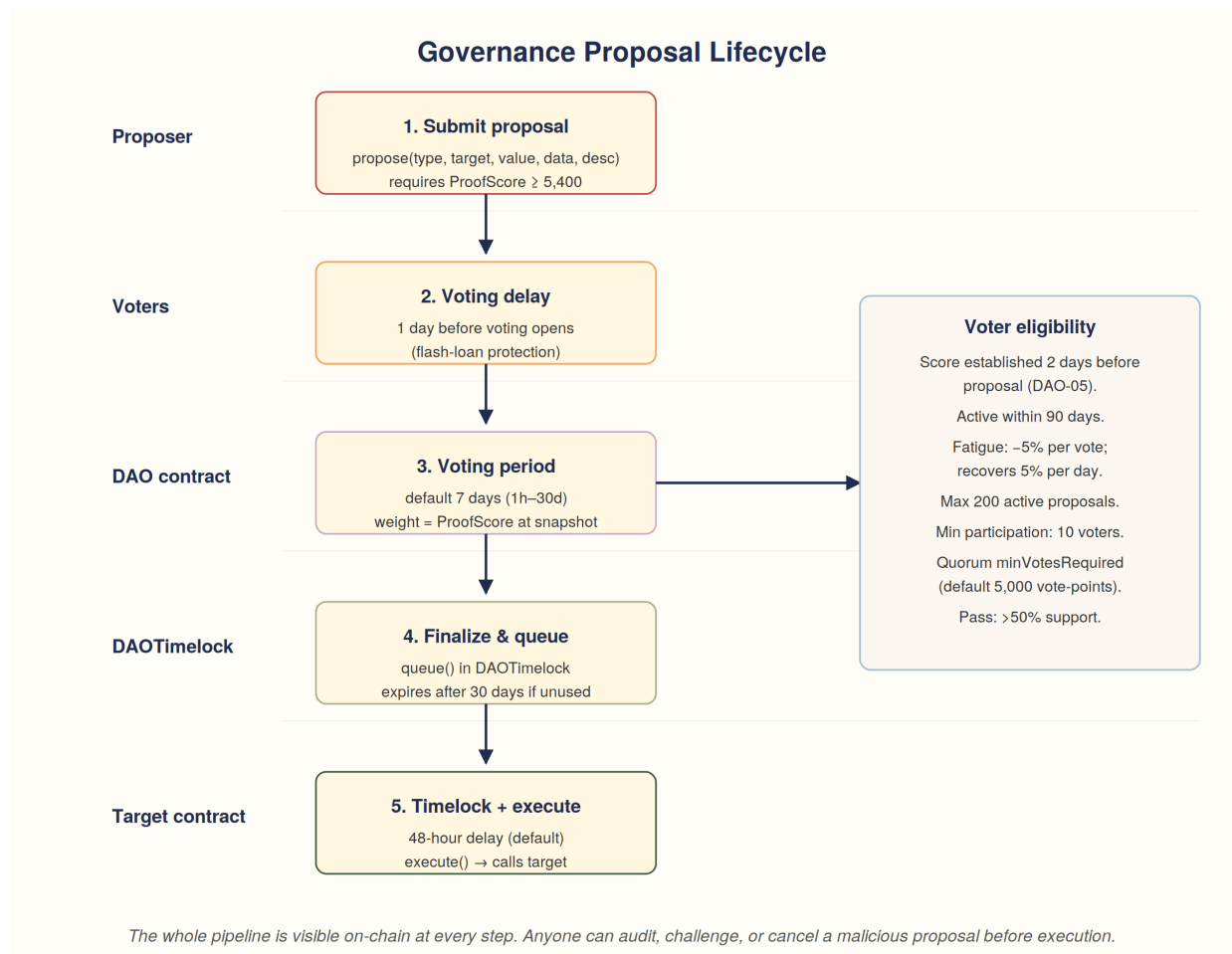
`CouncilManager` tracks each member's score and automatically removes anyone whose score has been below 7,000 for seven consecutive days (`GRACE_PERIOD`). The DAO can also remove a member for cause through a normal proposal.

`CouncilSalary` distributes salary to active council members every four months (`payInterval = 120` days) from the `DAOPayrollPool`. To receive salary, a member must have a current `ProofScore ≥ 7,000` (`minScoreToPay`). A `REINSTATE_DELAY` of seven days governs returning removed members. The salary is framed as employment compensation, not as investment return — this is a deliberate Howey-compliance design choice.

AdminMultiSig and the community veto

The `AdminMultiSig` is a *separate* 5-signer multisig from the elected governance council. Don't confuse them. The `AdminMultiSig` handles operational admin actions (3-of-5 for normal proposals, 4-of-5 for emergency). It has configurable timelock delays: 24 hours for config changes, 48 hours for critical changes, 1 hour for emergencies. Proposals expire after thirty days if not executed.

What makes the `AdminMultiSig` unusual is the **community veto**. Any user with `ProofScore ≥ 5,000` (`vetoMinScore`) and a minimum `VFIDE` stake (`vetoMinStake`, fallback 10,000 `VFIDE`) can vote to veto a pending `AdminMultiSig` proposal. If the veto threshold is reached during the timelock window, the proposal is killed. This gives the community a direct check on admin actions without needing to wait for the next governance cycle.



DAO proposal lifecycle. Each step is on-chain and visible. The 48-hour timelock between approval and execution is the community's reaction window.

TECHNICAL REFERENCE

6.3 Technical reference

Contracts

- **DAO** (1,129 LOC) — proposal lifecycle, voting, quorum rescue, fatigue, settlement window.
- **DAOTimelock** (539 LOC) — execution delay enforcer. Default 48 hours, `MIN_DELAY` 12 hours, `ABSOLUTE_MIN_DELAY` 24 hours.
- **AdminMultiSig** (533 LOC) — 5-signer multisig with community veto.
- **CouncilElection** (592 LOC) — twelve-seat council elections, fixed-term enforcement.
- **CouncilManager** (449 LOC) — score monitoring, automatic removals, term tracking.
- **CouncilSalary** (303 LOC) — quarterly salary distribution from `DAOPayrollPool`.
- **SystemHandover** (298 LOC) — six-month developer key burn.
- **GovernanceHooks** (240 LOC) — pre/post-action callbacks for governance events.

Critical constants

Constant	Value	Source
votingPeriod	7 days (default; $1h \leq p \leq 30d$)	DAO.sol:86
votingDelay	1 day (anti-flash-loan)	DAO.sol:89
MAX_PROPOSALS	200 active	DAO.sol:123
QUEUE_EXPIRY	30 days	DAO.sol:125
FATIGUE_PER_VOTE	5% per vote	DAO.sol:164
FATIGUE_RECOVERY_RATE	5% per day	DAO.sol:163
SCORE_SETTLEMENT_WINDOW	2 days	DAO.sol:167
EMERGENCY_RESCUE_DELAY	14 days	DAO.sol:97
BREAK_GLASS_DELAY	7 days	DAO.sol:77
DAOTimelock delay (default)	48 hours	DAOTimelock.sol:43
DAOTimelock MIN_DELAY	12 hours	DAOTimelock.sol:79
DAOTimelock ABSOLUTE_MIN_DELAY	24 hours	DAOTimelock.sol:107
councilSize (default)	12 (max 21)	CouncilElection.sol:97
FIXED_TERM_SECONDS	365 days (immutable)	CouncilElection.sol:25
FIXED_MAX_CONSECUTIVE_TERMS	1 (immutable)	CouncilElection.sol:24
COUNCIL_MIN_SCORE	7,000 (immutable)	CouncilManager.sol:56
GRACE_PERIOD (council removal)	7 days	CouncilManager.sol:54
payInterval (CouncilSalary)	120 days	CouncilSalary.sol:50
minScoreToPay (CouncilSalary)	7,000	CouncilSalary.sol:51
AdminMultiSig COUNCIL_SIZE	5	AdminMultiSig.sol:12
AdminMultiSig REQUIRED_APPROVALS	3	AdminMultiSig.sol:13
AdminMultiSig EMERGENCY_APPROVALS	4	AdminMultiSig.sol:14
AdminMultiSig CONFIG_DELAY	24 hours	AdminMultiSig.sol:16
AdminMultiSig CRITICAL_DELAY	48 hours	AdminMultiSig.sol:17
AdminMultiSig EMERGENCY_DELAY	1 hour	AdminMultiSig.sol:18
AdminMultiSig PROPOSAL_EXPIRY	30 days	AdminMultiSig.sol:20

Constant	Value	Source
SystemHandover monthsDelay	180 days (one 60d extension max)	SystemHandover.sol:49

ProposalType enum

```
enum ProposalType { Generic, Financial, ProtocolChange, SecurityAction }
```

Defined at DAO.sol:33. Each type has its own (target, selector) allow-list mapping (proposalTypeTargetAllowed, proposalTypeSelectorAllowed).

Key function signatures

```
function propose(ProposalType ptype, address target, uint256 value,
    bytes calldata data, string calldata description) external returns (uint256 id);

function vote(uint256 id, bool support) external;

function finalize(uint256 id) external;

function queue(uint256 id) external;

function execute(uint256 id) external;

function expireQueuedProposal(uint256 id) external;

// Emergency rescue:
function initiateEmergencyRescue() external onlyAdmin;
function approveEmergencyRescue() external; // initiator cannot self-approve
function executeEmergencyRescue() external; // after 14-day delay
```

Key events

ProposalCreated(id, proposer, ptype, target, value, data, description),
VoteCast(id, voter, support, weight, fatigue), ProposalQueued(id, eta),
ProposalExecuted(id), EmergencyQuorumRescueInitiated(readyAt),
QuorumProfileSynced(councilSize, minVotesRequired, minParticipation).

Code citations

Voting period: DAO.sol:86-88. Settlement window: DAO.sol:165-167. Activity requirement: DAO.sol:680. Fatigue: DAO.sol:160-164, 695-709. Emergency rescue: DAO.sol:336-385 (DAO-03 fix). Timelock delays: DAOTimelock.sol:43, 79, 107. Council fixed-term: CouncilElection.sol:24-25. Council min score + grace: CouncilManager.sol:54, 56, 177-191. Council salary: CouncilSalary.sol:50-51, 71. AdminMultiSig: AdminMultiSig.sol:12-20. SystemHandover: SystemHandover.sol:49, 53, 145-158.

Chapter 7 · Endorsements and the Social Layer

PLAIN ENGLISH

7.1 Vouching for someone — the trust network

Trust on VFIDE is mostly built through your own actions, but it can also be built through the people who vouch for you. If a respected community member endorses you, your ProofScore goes up — temporarily. The endorsement decays over time, and you have to keep earning trust on your own to stay where you are. The point is to give newcomers a head start without creating a system where reputation can be permanently bought or borrowed.

To endorse someone, you need a ProofScore of at least 7,000. That's a high bar. It means endorsements come from people who have been reliable for a long time. A new user who's been honest for two months cannot endorse anyone — they don't have the score yet. This protects the network from endorsement chains where low-trust users boost each other.

There's a separate mentorship system for people willing to take a more active role. A mentor (ProofScore $\geq 7,200$) can formally take on a mentee — a new user — and both of them earn ProofScore as the mentee hits milestones (their first ten transactions, their first dispute resolved cleanly, their first hundred VFIDE in volume). The pair-bond is on-chain. The mentor's reputation is at stake; if the mentee turns out to be fraudulent, the mentor's score takes a hit.

If someone endorses you and later regrets it, they can revoke the endorsement. If you stop being honest, the endorsement still ages out on its own. Either way, no endorsement is forever, and yours has to be earned again every time it decays.

If you remember nothing else

Endorsements are short-term boosts from trusted people. You need 7,000 to give one, 7,200 to mentor. Endorsements decay; you have to keep earning your own score. Mentorship pairs both people's reputations together — the mentor's reputation is at stake when the mentee acts.

HOW IT WORKS

7.2 How it works

Endorsements are managed by the SeerSocial contract, which extends the core Seer with social-trust primitives. When a user endorses another, an Endorsement struct is recorded: the endorser, the subject, a weight (configurable by DAO; defaults to a small fixed amount), and an expiry timestamp.

The endorsement contributes a bonus to the subject's *automated* ProofScore calculation, capped and decayed. As more endorsements accumulate, the bonus respects per-subject limits so a single user

cannot stack endorsements without bound. The cache (`cachedEndorsementExpiry`) tracks the earliest-expiring endorsement so the system knows when to recompute without scanning the whole list.

Endorsers face a cooldown (default `endorsementCooldown` = 1 day) between endorsements they give. A single user cannot endorse more than the configurable per-endorser limit at once. Endorsing carries some weight — once you endorse, the subject's behavior reflects on you a little, so endorsers have an interest in choosing carefully.

Mentorship is a separate pair-bond. `registerMentor` lets a high-score user become an available mentor. `sponsorMentee` creates the pair-bond. Once paired, the contract emits `MenteeSponsored` and tracks the relationship. As the mentee earns `ProofScore` through honest activity, both the mentor and mentee receive small reward credits. If the mentee gets penalized for fraud or default, the mentor's score takes a proportional hit. Either party can dissolve the mentorship at any time.

The DAO holds the keys to the social policy: endorsement weight, cooldown, validity period, mentor minimum score, and per-user limits are all configurable via `timelocked` governance. The hard floors (10-minute cooldown minimum, 7-day cooldown maximum) are coded as guardrails — the DAO cannot misconfigure them outside that range.

TECHNICAL REFERENCE

7.3 Technical reference

Contract

- **SeerSocial** (736 LOC, `contracts/future/SeerSocial.sol`) — endorsements, mentorship, and appeal extensions to the Seer engine.

Critical constants and configurable values

Name	Default	Source
<code>minScoreToEndorse</code>	7,000	<code>SeerSocial.sol:117</code>
<code>minScoreToMentor</code>	7,200	<code>SeerSocial.sol:127</code>
<code>endorsementCooldown</code>	1 day ($10 \text{ min} \leq c \leq 7 \text{ days}$)	<code>SeerSocial.sol:116, 238</code>
<code>endorsementValidity</code>	Configurable (DAO-set)	<code>SeerSocial.sol</code> policy
<code>maxEndorsementsPerSubject</code>	Configurable (DAO-set)	<code>SeerSocial.sol</code> policy
<code>maxEndorsementsPerEndorser</code>	Configurable (DAO-set)	<code>SeerSocial.sol</code> policy

Endorsement struct

```
struct Endorsement {
    address endorser;
    uint16 weight;
    uint64 expiry;
    string reason;
}

// Storage
mapping(address => mapping(address => Endorsement)) public endorsements;
// subject => endorser => Endorsement
```

Key function signatures

```
function endorse(address subject, string calldata reason) external;
function revokeEndorsement(address subject) external;
function registerMentor() external;
function revokeMentorStatus() external;
function sponsorMentee(address mentee) external;
function dissolveMentorship(address mentee) external;

// Score-bonus computation
function calculateEndorsementBonus(address subject) external view returns (uint16);
function pruneExpiredEndorsements(address subject, uint16 maxToProcess) external;
```

Key events

UserEndorsed(endorser, subject, weight, expiry, reason),
EndorsementRevoked(endorser, subject, weight, reason),
EndorserPruned(endorser, endorsementsCleaned), MentorRegistered(mentor),
MentorRevoked(mentor), MenteeSponsored(mentor, mentee), MenteeRemoved(mentor,
mentee).

Code citations

Endorse threshold: SeerSocial.sol:117. Mentor threshold: SeerSocial.sol:127. Cooldown range:
SeerSocial.sol:116, 238. Endorsement struct + storage: SeerSocial.sol:90-105. Cache machinery:
SeerSocial.sol:101-108 (F-36 fix to avoid O(n) SLOADs).

Chapter 8 · Lending and Borrowing

PLAIN ENGLISH

8.1 Borrow without a credit score

VFIDE has direct peer-to-peer lending. Anyone can be a lender. Anyone can be a borrower. Two strangers can write a loan agreement on-chain in a few minutes. There is no bank, no underwriter, no credit bureau. The borrower's ProofScore is the credit history, and that history is owned by the borrower — portable, verifiable, and impossible to take away.

The lender posts an offer: amount, interest rate (capped at 12% per term), duration (1 to 30 days), and a minimum borrower ProofScore. The borrower accepts. If the borrower has guarantors — co-signers whose vaults back part of the loan — they can include those too, which usually unlocks better terms or larger amounts. Repayment supports a single payment or a structured plan with installments.

If the borrower repays on time, both the lender and the borrower earn a small ProofScore boost. The borrower's reputation grows, which means easier borrowing next time. If the borrower is late, they take a small score hit. If they default completely, the hit is bigger, and the guarantors take score hits too — that's the cost of co-signing for someone you don't fully trust.

There is also a separate **flash loan** system for power users. A flash loan is a loan you take out and repay in the same transaction. If you don't pay it back at the end of the transaction, the whole thing reverts and nothing happens. Flash loans are useful for things like arbitrage between two markets, refinancing, or atomic position changes. They are not relevant for ordinary users.

If you remember nothing else

Lending is direct peer-to-peer. ProofScore is your credit history. On-time repayment raises both lender's and borrower's scores; default penalizes the borrower and any guarantors. Flash loans exist for sophisticated users only.

HOW IT WORKS

8.2 How it works

Term loans

VFIDETermLoan manages a marketplace of lender offers and borrower acceptances. A user posts a lender offer with amount, max interest rate (capped at 1,200 bps = 12%), duration (1 to 30 days), required minimum ProofScore, and whether guarantors are required. Other users browse the offers and accept the ones that match their needs.

When a borrower accepts an offer, the contract checks that the borrower's score meets the lender's minimum, that the borrower has fewer than ten active loans (MAX_ACTIVE_LOANS), and that any required guarantors have agreed to co-sign. Co-signing has a fourteen-day window before the offer

expires (`COSIGNING_TIMEOUT`); if the guarantors don't act, the loan never starts.

Repayment options:

- **Single payment.** The borrower repays the principal plus interest at maturity. If they're late, a 3-day grace period (`GRACE_PERIOD`) softens the penalty.
- **Installment plan.** The borrower commits to a payment plan up to 90 days long with up to 12 installments. Plans completed on time count as good credit; plans abandoned mid-stream incur the highest score penalties in the system.
- **Revenue assignment.** If the borrower defaults, the contract can extract a percentage (10%) of their daily incoming VFIDE volume until the debt is repaid. Daily extraction interval prevents draining the borrower's life — they keep most of what comes in but have a portion redirected to the lender.

ProofScore consequences are explicit and tier-graduated. A successful repayment is a small reward (+0.5% borrower, +0.2% lender). Late but paid is a small penalty (-1.5%). A failed plan is a tougher penalty (-10% then -5% if abandoned, total -15%). A full default gets -10% twice for a total of -20%. Guarantors take -5% per failed plan or -10% per full default, distributed across all guarantors.

10% of all interest paid (`PROTOCOL_CUT_PCT = 10`) goes to the community ecosystem fund. The remaining 90% goes to the lender. This is the only way the protocol takes a piece — and it's a piece of the interest, not the principal, so the lender's capital is never reduced by protocol overhead.

Flash loans

VFIDEFlashLoan implements ERC-3156 — the standard flash-loan interface. Each lender deposits their own VFIDE, sets their own fee rate (within protocol caps), and can pause or withdraw at any time. The borrower selects a specific lender, executes via `flashLoan(lender, receiver, amount, maxFeeBps, data)`, and the receiver's `onFlashLoan()` callback executes the strategy — typically arbitrage, collateral swap, or refinancing.

If the receiver doesn't return amount + fee at the end of the transaction, the whole thing reverts and no state changes persist. Atomicity is guaranteed by the EVM. The fee split is 90% to the lender, 10% to the FeeDistributor. There is an anti-spam cooldown between loans from the same borrower.

Both deposit and flashLoan are gated on `systemExemptConfirmed` — a flag that VFIDEToken sets after the FlashLoan contract has been registered as a system-exempt contract. This is a deploy-time race fix; the contract cannot operate before its exemption is confirmed (line 249).

TECHNICAL REFERENCE

8.3 Technical reference

Contracts

- **VFIDETermLoan** (1,139 LOC) — peer-to-peer term lending.
- **VFIDEFlashLoan** (644 LOC) — ERC-3156 flash loans.

VFIDETermLoan critical constants

Name	Value	Meaning
MAX_INTEREST_BPS	1,200 (12%)	Hard cap on interest rate per term
MAX_DURATION	30 days	Maximum loan length
MIN_DURATION	1 day	Minimum loan length
GRACE_PERIOD	3 days	Late-but-paid window
MAX_ACTIVE_LOANS	10	Per borrower
MAX_PLAN_DURATION	90 days	Installment plan max length
MAX_PLAN_INSTALLMENTS	12	Installment plan max payment count
GUARANTOR_LIABILITY_PCT	100	Guarantors collectively cover 100% of principal
EXTRACTION_INTERVAL	1 day	Daily revenue extraction after default
EXTRACTION_RATE_PCT	10	Percent of daily incoming volume extracted
COSIGNING_TIMEOUT	14 days	Window for guarantors to co-sign
PROTOCOL_CUT_PCT	10	Portion of interest going to ecosystem

ProofScore consequences

Outcome	Borrower	Lender	Guarantors
On-time repayment	+0.5% (REWARD_ONTIME_BORROWER = 5)	+0.2% (REWARD_ONTIME_LENDER = 2)	—
Late but paid	-1.5% (PENALTY_LATE = 15)	—	—
Plan completed	-3.0% (PENALTY_PLAN_COMPLETED = 30, good-faith ack)	+0.2%	—
Plan failed	-10% then -5% (-15% total)	—	-5% each (PENALTY_PLAN_FAILED_GUARANTOR = 50)
Full default	-10% then -10% (-20% total)	—	-10% each (PENALTY_DEFAULT_GUARANTOR = 100)

VFIDEFlashLoan key facts

ERC-3156 compliant. Callback signature: `onFlashLoan(initiator, token, amount, fee, data)` returns `(bytes32)`. Successful callback must return `keccak256("ERC3156FlashBorrower.onFlashLoan")`. Fee split: 90% lender, 10% FeeDistributor. Lender deposits and flashLoan are gated on `systemExemptConfirmed`.

Code citations

VFIDETermLoan constants: VFIDETermLoan.sol:117-156. Reward/penalty application: VFIDETermLoan.sol:536-540 (on-time), 724-740 (failures). Revenue assignment: VFIDETermLoan.sol:127-130. VFIDEFlashLoan ERC-3156: VFIDEFlashLoan.sol:53-54, 108. FlashLoan systemExempt gate: VFIDEFlashLoan.sol:249.

Chapter 9 · Badges and Achievements

PLAIN ENGLISH

9.1 Visible reputation

Every user can earn badges. Badges are the visible part of your reputation — little markers next to your name that say what you've done. Some are about being early. Some are about being a great merchant. Some are about helping the network. Some are about being honest in disputes. Most of them give you a small ProofScore boost that stacks with the rest of your trust.

Badges are **soulbound** — they cannot be transferred, sold, or moved. They belong to the wallet that earned them. If you give away your wallet, the badges stay with the wallet, not with you. This makes them genuine markers of behavior rather than tradable status symbols.

There are twenty-eight badge types. Some are automatic (the system grants them when you meet the conditions). Some are claimed (you have to take an action to receive them). Some are awarded by the DAO for specific contributions like bug reports or translations. Each badge has a different ProofScore weight ranging from 15 to 60 points — small individually, meaningful collectively.

Two badges are scarce. **Pioneer** goes to the first 10,000 users. **Founding Member** goes to the first 1,000 users to reach a ProofScore of 800 or higher. Once those slots are filled, that's it. Other badges are available to anyone who qualifies, with no scarcity cap.

If you remember nothing else

Badges are achievement markers. They are soulbound — non-transferable. They give small ProofScore boosts (+15 to +60 each). 28 types. Some are first-come (Pioneer, Founding Member). Most are earned by behavior.

HOW IT WORKS

9.2 How it works

Badges are managed by three contracts working together. `BadgeRegistry` is the catalog: it defines all 28 badge constants, their human-readable names, their ProofScore weights, and which (if any) have a renewal period. `BadgeManager` is the qualification engine: it watches user behavior and either grants badges automatically or processes claims when users request a badge they qualify for. `VFIDEBadgeNFT` is the on-chain representation: each earned badge is minted as an ERC-721 token following the ERC-5192 soulbound standard, which means the token is permanently locked to the wallet that minted it.

When a user qualifies for a badge, `BadgeManager` calls `seer.awardBadge(user, badgeId)` on the `Seer` contract, which records the badge in the user's profile and contributes the badge's weight to the ProofScore calculation. Some badges are time-limited and need to be renewed (active-trader

badges expire after 90 days, governance-voter badges after 180 days). Renewal happens automatically when the user keeps doing the qualifying behavior.

The 28 badges

Badge	Weight	Earn condition (summary)
PIONEER	+30	First 10,000 users
FOUNDING_MEMBER	+50	First 1,000 users to reach ProofScore $\geq$ 800 (capped at MAX_FOUNDING_MEMBERS = 1,000)
EARLY_TESTER	+25	Active during testnet, completed test transactions
ACTIVE_TRADER	+20	Sustained transaction volume; expires after 90 days, renewable
GOVERNANCE_VOTER	+25	Voted in DAO proposals; expires after 180 days, renewable
POWER_USER	+40	Heavy use across multiple protocol surfaces
DAILY_CHAMPION	+15	Most-active user on a given day
TRUSTED_ENDORSER	+30	Endorsements you gave proved sound (subjects didn't get penalized)
COMMUNITY_BUILDER	+35	Onboarded multiple new users
PEACEMAKER	+25	Resolved disputes amicably without escalation
MENTOR	+30	Successfully mentored newcomers via SeerSocial
VERIFIED_MERCHANT	+40	Registered merchant with active transactions
ELITE_MERCHANT	+60	High-volume merchant, zero disputes
INSTANT_SETTLEMENT	+20	Merchant earned 8,000+ trust $\rightarrow$ 3-day escrow
ZERO_DISPUTE	+25	Maintained spotless dispute record
FRAUD_HUNTER	+50	Filed valid fraud reports that the DAO confirmed
CLEAN_RECORD	+20	Long-standing account with no penalties
REDEMPTION	+30	Recovered ProofScore after a penalty
GUARDIAN	+40	Active guardian on multiple vaults; rotation/recovery actions
ELITE_ACHIEVER	+50	Multiple high-tier badges earned
CENTURY_ENDORSER	+35	100+ valid endorsements given
WHALE_SLAYER	+25	Voted against a proposal favored by token whales
DIVERSIFICATION_MASTER	+30	Used multiple protocol features (lending, escrow, governance)
EDUCATOR	+30	Created educational content for the network
CONTRIBUTOR	+40	DAO-recognized contribution
BUG_BOUNTY	+50	Submitted valid security report (variable in practice)
TRANSLATOR	+25	Contributed translations for the localization system
HEADHUNTER	+35	Top quarterly recruiter (top-20 each quarter)

All badges are soulbound — the ERC-5192 `locked(uint256 tokenId)` view returns true unconditionally (`VFIDEBadgeNFT.sol:190`). The `_update` hook prevents any transfer outside of the minting and burning paths.

TECHNICAL REFERENCE

9.3 Technical reference

Contracts

- **BadgeRegistry** (335 LOC, `contracts/future/BadgeRegistry.sol`) — catalog. 28 bytes32 public constant badge IDs.
- **BadgeManager** (591 LOC, `contracts/future/BadgeManager.sol`) — qualification engine and award flow.
- **VFIDEBadgeNFT** (475 LOC, `contracts/future/VFIDEBadgeNFT.sol`) — ERC-721 + ERC-5192 soulbound representation.
- **BadgeQualificationRules** (92 LOC) — qualification logic library.

Key constants

```
// BadgeRegistry – 28 badge IDs (excerpt)
bytes32 public constant PIONEER = keccak256("PIONEER");
bytes32 public constant FOUNDING_MEMBER = keccak256("FOUNDING_MEMBER");
// ... full list at BadgeRegistry.sol

// BadgeManager scarcity caps
MAX_FOUNDING_MEMBERS = 1_000 (line 66)

// VFIDEBadgeNFT timelocks
pendingBaseTokenURIat + base URI timelock
```

Key function signatures

```
// BadgeRegistry views
function isValidBadge(bytes32 badge) public pure returns (bool);
function getRecommendedWeight(bytes32 badge) public pure returns (uint16);
function getName(bytes32 badge) public pure returns (string memory);

// BadgeManager actions
function awardBadge(address user, bytes32 badge) external onlyOperator;
function claimBadge(bytes32 badge) external; // for self-claimable badges
function checkAndAward(address user) external; // re-evaluate qualifications

// VFIDEBadgeNFT (ERC-5192)
function locked(uint256 tokenId) external pure returns (bool); // always true
```

Key events

`BadgeAwarded(user, badge, weight)`, `BadgeRevoked(user, badge, reason)`, `BadgeRenewed(user, badge, newExpiry)`, `Locked(tokenId)` (ERC-5192 standard event), `Transfer(0, owner, tokenId)` (ERC-721 mint).

Code citations

Badge constants list: BadgeRegistry.sol (28 lines starting at the bytes32 constants block). Weight returns: BadgeRegistry.sol:284+ (getRecommendedWeight). Founding-member cap: BadgeManager.sol:66 (MAX_FOUNDING_MEMBERS = 1_000), 495-500 (auto-award). Soulbound enforcement: VFIDEBadgeNFT.sol:184-200 (ERC-5192 _update hook + locked view), 471 (interface ID).

Chapter 10 · Disputes, Fraud Reports, and Appeals

PLAIN ENGLISH

10.1 What to do when something goes wrong

VFIDE has three different paths for resolving conflicts, and the right one depends on what kind of conflict it is.

Escrow disputes happen when you've used the EscrowManager for a high-stakes trade — a buyer who paid for goods that didn't arrive, or a merchant whose payment is held in escrow and the buyer is refusing to release it. EscrowManager has a built-in arbiter system. For ordinary amounts, a neutral arbiter (someone who is neither buyer nor merchant) decides. For amounts above 10,000 VFIDE, the dispute goes to the DAO — twelve council members deciding together rather than one person. The loser takes a ProofScore penalty. There are no chargeback fees.

Fraud reports are for accusations of bad-faith behavior outside an escrow context — a merchant taking payment and never shipping, an obvious scammer pattern. Anyone with ProofScore $\geq 5,000$ can file a fraud complaint against another user. Three independent complaints flag the user for DAO review (not auto-flagging — three reports trigger a process, not an automatic punishment). The DAO reviews evidence and either confirms fraud or dismisses. Confirmed fraud activates a thirty-day transfer escrow: the user's outgoing transfers are held for thirty days then delivered. Tokens are never seized — just delayed, so victims have time to file claims and reverse damage.

Score appeals are for disputing a ProofScore reduction. If you think an operator or the DAO penalized you unfairly, you have the right to file a formal review (it's the Right to Challenge in the Seer Constitution). The DAO reviews the evidence. Equivalent actions must result in equivalent treatment — that's a constitutional principle, not a courtesy.

All three paths work because nobody is in a hurry to enforce blindly. The ProofScore consequences for losing a dispute are real but bounded; rehabilitation is always possible (Right to Recovery). And every step leaves an on-chain record, so patterns of bad-faith filing or repeated abuse can themselves be flagged.

If you remember nothing else

Escrow disputes use the arbiter (or DAO above 10,000 VFIDE). Fraud reports flag after 3 independent complaints and trigger DAO review. Score appeals exercise your Right to Challenge. Tokens are never seized — confirmed fraud just delays transfers by 30 days so victims can recover. Penalties are bounded; recovery is always possible.

HOW IT WORKS

10.2 How it works

Escrow disputes

When two parties enter an escrow trade, they agree on (1) the amount, (2) the lock period (which adapts to the seller's ProofScore — see Chapter 5), and (3) which arbiter is named for dispute resolution. The arbiter must not be either party (conflict-of-interest enforced in code). Arbiter changes require a 7-day timelock (`ARBITER_TIMELOCK`).

If neither party disputes during the lock period, the escrow auto-releases to the merchant. If the buyer disputes (e.g., goods didn't arrive), the escrow freezes and waits for arbiter resolution. The arbiter calls `resolveDispute(escrowId, buyerWins, buyerShareBps)` with a basis-points share for the buyer (0-10000). The split executes automatically. The losing party takes a ProofScore penalty proportional to the loss; this makes future fraud expensive.

For amounts above `HIGH_VALUE_THRESHOLD = 10,000 VFIDE`, disputes route to the DAO instead of the named arbiter. Twelve council members deciding together is more robust than one person. There is also a 90-day timeout: if the DAO doesn't respond, the buyer can self-resolve to prevent indefinite freezing of funds.

Fraud reports

`FraudRegistry` implements community-driven fraud protection without giving anyone the power to seize tokens. Filing a complaint requires the filer to have ProofScore $\geq 5,000$ — this prevents new accounts from spamming fraud reports. The complaint is recorded with the target, reason, and filer.

Each target has a complaint counter (`complaintCount[target]`). At three complaints (`COMPLAINTS_TO_FLAG`), the target enters `isPendingReview` status — flagged for DAO review but not yet penalized. The DAO reviews evidence (typically off-chain, with IPFS hashes for documentation) and either confirms fraud or dismisses. If dismissed, complaint counts reset; if confirmed, the target moves to `isFlagged`.

A flagged user's outgoing VFIDE transfers are routed through a thirty-day escrow (`ESCROW_DURATION = 30 days`). Their tokens are not seized — they will be delivered, eventually — but the thirty-day delay gives victims time to file civil claims and the DAO time to investigate further. The DAO can clear flags (rehabilitation), extend them, or in extreme cases escalate. There is no path to permanently confiscate tokens; the protocol's non-custodial guarantee holds even for confirmed fraudsters.

Score appeals

Any user can call `Seer.requestScoreReview(reason)`. The request enters a queue for the DAO. The DAO reviews (typically with off-chain evidence) and calls `resolveScoreDispute(subject, approved, adjustment)`. If approved, the score is adjusted by the proposed amount; if rejected, no change.

More structurally, the Seer Constitution defines five rights that constrain what the protocol can do to your score:

- **Right to reason.** Every score change emits a `ScoreChanged` event with a reason code.

- **Right to challenge.** The dispute mechanism above. Reachable by any subject.
- **Right to bounded penalties.** The cross-operator daily cap of 3% (300 points) is immutable.
- **Right to equal treatment.** Score-source weights and operator limits must apply uniformly.
- **Right to recovery.** Decay always moves toward neutral 5,000, never toward zero. Penalties expire. Score can always be earned back.

TECHNICAL REFERENCE

10.3 Technical reference

Contracts

- **EscrowManager** (433 LOC) — escrow lifecycle, arbiter resolution, DAO escalation.
- **FraudRegistry** (778 LOC) — community fraud reports, 30-day transfer escrow on confirmed fraud.
- **Seer** (1,403 LOC) — score-review and score-dispute paths.

Critical constants

Constant	Value	Source
EscrowManager HIGH_VALUE_THRESHOLD	10,000 × 10 ⁴ ■ VFIDE	EscrowManager.sol:89
EscrowManager ARBITER_TIMELOCK	7 days	EscrowManager.sol:92
EscrowManager dispute timeout (DAO non-response)	90 days	EscrowManager dispute path
FraudRegistry COMPLAINTS_TO_FLAG	3	FraudRegistry.sol (constant + check at 215)
FraudRegistry minScoreToFile	5,000	FraudRegistry filer requirement
FraudRegistry ESCROW_DURATION	30 days	FraudRegistry.sol:66
Seer DAO_SCORE_COOLDOWN	4 hours	Seer.sol:177
Seer maxDAOScoreChange	500 (5%, post H-5 fix)	Seer.sol:172

Key function signatures

```
// EscrowManager
function createEscrow(address merchant, uint256 amount, address arbiter, ...) returns
(uint256);
function release(uint256 escrowId) external;
function dispute(uint256 escrowId, string reason) external;
function resolveDispute(uint256 escrowId, bool buyerWins, uint256 buyerShareBps)
external onlyArbiter;
function escalateToDAO(uint256 escrowId) external; // for amounts >
HIGH_VALUE_THRESHOLD
function selfResolveAfterTimeout(uint256 escrowId) external; // 90-day fallback

// FraudRegistry
function fileComplaint(address target, string reason, bytes32 evidenceHash) external;
function reviewComplaint(address target, bool fraudConfirmed) external onlyDAO;
function clearFlag(address target, string reason) external onlyDAO;
function rescueExcessTokens(address user, address recipient) external onlyDAO;

// Seer score appeals
function requestScoreReview(string reason) external;
function resolveScoreDispute(address subject, bool approved, int256 adjustment)
external onlyDAO;
```

Key events

EscrowManager: EscrowCreated, EscrowFunded, EscrowReleased, EscrowDisputed, EscrowDisputeResolved. FraudRegistry: ComplaintFiled, PendingDAOResult, FraudConfirmed, FraudFlagCleared, EscrowReleased. Seer: ScoreReviewRequested, ScoreDisputeResolved, ScoreChanged (with reason code on every change).

Code citations

Escrow tiers and arbiter timelock: EscrowManager.sol:89, 92. Three-complaint flag threshold: FraudRegistry.sol:215. 30-day transfer escrow: FraudRegistry.sol:66, 249. Complaint reset on dismissal: FraudRegistry.sol:419, 477 (prevents stale-count abuse). Score review: Seer.sol requestScoreReview / resolveScoreDispute. Constitutional rights enforced: Seer.sol cross-operator cap (line 169) and DAO change limit (line 172).

Part IV

Technical Reference

T1 · Smart Contract Reference

This section catalogs every deployable contract, its role, its constructor and key public functions, and where the constants live. Contracts are grouped by deployment phase so the reading order matches the system's actual dependency graph.

Inventory at a glance

Group	Count	Description
Production	41	Top-level contracts/*.sol — Phase 1 and 2
Future	20	contracts/future/*.sol — Phase 3+
Interfaces	27	contracts/interfaces/*.sol
Mocks	11	Test-harness mocks
Pools	3	DAOPayrollPool, MerchantCompetitionPool, HeadhunterCompetitionPool
Libraries	3	ScoringConstants, Uniswap V3 FullMath/TickMath
Legacy	2	Forwarder shims for backward-compatible imports
Testnet	1	VFIDETestnetFaucet
TOTAL	108	Solidity files total. ~40,000 LOC.

Phase 1 contracts (deployed at testnet launch)

These eighteen contracts are deployed by `scripts/deploy-all.ts` in the order shown. The order matters because later contracts read addresses of earlier ones at construction time. After deploy, a 48-hour timelock window passes, then `scripts/apply-all.ts` wires the cross-references.

#	Contract	LOC	Role
1	ProofLedger	121	Append-only audit log used by token, vault, score events.
2	DevReserveVestingVault	335	50M VFIDE vesting (60-day cliff, 30 bi-monthly unlocks).
3	VFIDEToken	1,454	ERC-20 with custom <code>_transfer</code> hooking <code>BurnRouter</code> and anti-whale.
4	Seer	1,403	ProofScore engine with 50-entry history buffer per user.
5	ProofScoreBurnRouter	1,011	Fee curve, microtx ceiling, 40/10/50 split.
6	VaultHub	638	CardBoundVault factory + registry.
7	DAOPayrollPool	66	Council salary recipient pool.

#	Contract	LOC	Role
8	MerchantCompetitionPool	90	Merchant rewards pool.
9	HeadhunterCompetitionPool	70	Onboarding rewards pool.
10	FeeDistributor	500	Ecosystem fee redistribution (35/20/15/20/10).
11	MerchantPortal	1,217	Merchant registration, dispute, fees=0.
12	DAOTimelock	539	48h-default execution delay.
13	GovernanceHooks	240	Pre/post-action governance callbacks.
14	DAO	1,129	Proposal lifecycle, ProofScore-weighted voting.
15	VFIDEFlashLoan	644	ERC-3156 flash loans.
16	VFIDETermLoan	1,139	Peer-to-peer term lending.
17	FraudRegistry	778	Community fraud reports + 30-day escrow.
18	VFIDETestnetFaucet	394	Testnet-only free VFIDE + ETH faucet.

Phase 2 contracts (vault recovery, council, system handover)

Contract	LOC	Role
CardBoundVault	1,536	Per-user smart-contract wallet (deployed by VaultHub via CREATE2).
CardBoundVaultDeployer	147	CREATE2 deployer helper for deterministic addresses.
VaultRegistry	903	Recovery-id (hashed email/phone) to vault mapping for lookup.
VaultRecoveryClaim	760	14-day guardian vote + 7-day challenge recovery flow.
VaultInfrastructure	1,333	Legacy vault migration helpers + bytecode provider.
AdminMultiSig	533	5-signer admin multisig with community veto.
SystemHandover	298	6-month developer key burn timer.
EmergencyControl	634	Foundation rotation and panic pause coordinator.
OwnerControlPanel	1,326	Post-handover admin surface — deliberately omits setFrozen.
VFIDEAccessControl	186	Role-based access primitives.

Phase 3+ — Commerce, Bridge, Social

Contract	LOC	Role
EscrowManager	433	High-stakes escrow with arbiter resolution.
VFIDECommerce	411	Storefront helpers.
MainstreamPayments	1,432	POS terminal registry, multi-currency router (future).
SubscriptionManager	616	Recurring billing for merchants (future).
PayrollManager	761	Staff payroll for merchants employing others.
SeerSocial	736	Endorsements + mentorship (future).
SeerAutonomous	1,320	Pattern-detection score adjustments (future).
SeerGuardian	648	DAO ↔ Seer mutual oversight.
SeerWorkAttestation	436	Verified-work reward attestations.
SeerView	232	Read-only Seer query helpers.
SeerPolicyGuard	213	Score-source policy enforcement.
BadgeManager	591	Badge qualification engine.
BadgeRegistry	335	28-badge catalog with weights.
BadgeQualificationRules	92	Qualification logic library.
VFIDEBadgeNFT	475	ERC-5192 soulbound badge tokens.
VFIDEBridge	1,168	LayerZero OFT cross-chain bridge.
BridgeSecurityModule	334	Bridge rate limits and replay protection.
VFIDEBenefits	210	Tier-based feature unlocks.
VFIDEEnterpriseGateway	445	Enterprise feature surface.
CouncilElection	592	Council nominations and elections.
CouncilManager	449	Council member status + auto-removal.
CouncilSalary	303	Quarterly council salary distribution.

Treasury and pools

Contract	LOC	Role
EcosystemVault	1,545	Receives ecosystem 50% share, redistributes via FeeDistributor.
EcosystemVaultLib	110	Math helpers for EcosystemVault.
EcosystemVaultView	247	Read-only views into EcosystemVault state.
SanctumVault	564	Charity disbursement vault (24h delay, 90-day proposal expiry).
RevenueSplitter	153	Generic revenue-split helper used by treasury flows.
VFIDEFinance	262	Treasury-management primitives.
DAOPayrollPool	66	Salary recipient pool.
MerchantCompetitionPool	90	Merchant rewards pool.
HeadhunterCompetitionPool	70	Onboarding rewards pool.
LiquidityIncentives	270	DEX liquidity coordination — zero rewards by design (Howey).
DutyDistributor	133	Non-monetary DAO participation tracking (Duty Points).
ServicePool	495	Generic service-payment pool.

Oracles and safety

Contract	LOC	Role
VFIDEPriceOracle	603	Hybrid Chainlink + Uniswap V3 TWAP price oracle.
StablecoinRegistry	308	DAO-managed stablecoin allow-list.
CircuitBreaker	556	Auto-pause on suspicious volume / price drops.
SharedInterfaces	562	Local OZ-equivalent primitives (avoid npm supply chain).

Per-contract function signatures, events, errors, and constants are documented inline in the prior chapters' Technical Reference sections, where each contract appears in its functional context. The constants reference card at the back of the manual (Appendix 4) consolidates every threshold value in one place for quick lookup.

T2 · API Reference

VFIDE has 127 API routes total under `app/api/`, organized by domain. Authentication is JWT-based with HTTP-only secure cookie fallback (`verifyAuth` middleware). Rate limiting uses Upstash Redis (`withRateLimit`). All write operations require CSRF token validation (double-submit cookie pattern). Content-type validation prevents MIME confusion. All inputs are validated via Zod schemas at route boundaries.

Domain map

Domain	Routes	Purpose
auth/	challenge/response wallet auth	Sign-In with Ethereum (SIWE) flow.
merchant/ (28 sub-routes)	Catalogued in Chapter 5.3	Full merchant operating system.
groups/	messaging primitives	Encrypted group messaging.
community/	posts, stories, feed	Social feed and public posts.
crypto/	price, fees, exchange rate	Public read-only crypto queries.
badges/	metadata, claim	Badge catalog and claiming.
notifications/	fetch, mark-read	User notification feed.
proposals/	DAO proposals	Read DAO proposals and votes.
endorsements/	endorse, revoke, list	SeerSocial endorsements surface.
flashloans/	lender registration, offer	VFIDEFlashLoan UI surface.
leaderboard/	rankings	Top users by ProofScore, volume.
referral/	track, claim	Referral incentives.
remittance/	transfer flow	Cross-border remittance UX.
security/	report, alerts	Security incident reports.
seer/	score query, history	Public ProofScore lookups.
transactions/	history, status	User transaction feed.
media/	upload, fetch	Media attachments (CDN-backed).
account/	profile, settings	User profile management.
health/	heartbeat	Operational health checks.
webhooks/	outbound delivery	Webhook delivery from merchants.

Common request/response shape

Every authenticated POST/PUT/PATCH/DELETE expects:

Headers:

Authorization: Bearer <jwt> (or HTTP-only cookie)

Content-Type: application/json
X-CSRF-Token: <token> (must match cookie)

Body (validated by Zod):
{ ...domain-specific JSON... }

Response:
200/201: { data: {...}, meta: { requestId, ... } }
400: { error: { code, message, fieldErrors? } }
401: unauthenticated
403: authenticated but forbidden
429: rate limited
500: server error (logged with requestId)

Every API route lives at `app/api/<domain>/<sub>/route.ts`. The complete enumeration of routes can be reproduced with `find app/api -name route.ts | wc -l` (yields 127 in v19.13).

T3 · Database Schema

VFIDE uses PostgreSQL with Row-Level Security (RLS) policies applied to all user-data tables. The schema is managed via 136 SQL migration files in `/migrations/`. Per-connection user context is set via `SELECT set_config('app.current_user_address', $1, false)`, which RLS policies use as the gate for row-level access.

Core tables (representative — full schema in migrations/)

Table	Purpose
user_profiles	Wallet address, username, avatar URL, bio, locale preference.
sessions	JWT session tracking, refresh tokens.
messages	Encrypted direct/group messages.
groups, group_members	Encrypted group chats.
notifications	Per-user notification feed.
merchant_profiles	Off-chain merchant metadata, business info.
invoices, invoice_lines	Merchant invoices with line items.
products, product_variants, inventory	Merchant inventory tracking.
customers, customer_segments	Merchant CRM.
loyalty_programs, loyalty_balances	Per-merchant loyalty programs.
bookings, booking_slots	Appointment scheduling.
coupons, coupon_redemptions	Discount codes.
transaction_metadata	Off-chain enrichment of on-chain transactions.
community_posts, community_stories	Public social feed content.
fraud_complaints_offchain	Off-chain evidence for FraudRegistry filings.
badge_metadata	Off-chain badge images, descriptions, IPFS hashes.
dispute_evidence	Off-chain evidence for escrow disputes.
referrals, referral_codes	Headhunter pool tracking.
audit_log	Append-only audit trail of admin actions.

Row-Level Security pattern

```
-- Example: merchant_profiles
ALTER TABLE merchant_profiles ENABLE ROW LEVEL SECURITY;

CREATE POLICY merchant_owner_read ON merchant_profiles
  FOR SELECT USING (wallet_address =
current_setting('app.current_user_address')::text);

CREATE POLICY merchant_owner_write ON merchant_profiles
  FOR UPDATE USING (wallet_address =
current_setting('app.current_user_address')::text);
```

Backend lib/db.ts is responsible for issuing `SET app.current_user_address` on every connection checkout from the pool. If the pattern is missed (or if a developer uses `SET LOCAL` incorrectly), RLS would over-allow and leak data. This is one of the recurring audit findings to recheck before mainnet.

T4 · Configuration Reference

All environment variables consumed by the frontend, the WebSocket server, or the deploy scripts. Each is read at boot time; missing required values fail fast (scripts/postinstall-validate-env.cjs).

Frontend / Next.js

Var	Purpose
NEXT_PUBLIC_IS_TESTNET	Switch UI affordances (faucet visible, network selector).
NEXT_PUBLIC_BASE_RPC_URL	Base mainnet RPC endpoint.
NEXT_PUBLIC_BASE_SEPOLIA_RPC_URL	Base Sepolia RPC endpoint.
NEXT_PUBLIC_VFIDE_TOKEN_ADDRESSES	Deployed VFIDEToken address.
NEXT_PUBLIC_VAULT_HUB_ADDRESS	Deployed VaultHub address.
NEXT_PUBLIC_SEER_ADDRESS	Deployed Seer address.
NEXT_PUBLIC_DAO_ADDRESS	Deployed DAO address.
NEXT_PUBLIC_BURN_ROUTER_ADDRESSES	Deployed ProofScoreBurnRouter.
NEXT_PUBLIC_FAUCET_ADDRESS	VFIDEtestnetFaucet (testnet only).
NEXT_PUBLIC_WEBSOCKET_URL	WebSocket server URL.
DATABASE_URL	PostgreSQL connection string.
UPSTASH_REDIS_URL	Rate-limit Redis.
JWT_SECRET	JWT signing key (server-only).
CSRF_SECRET	CSRF token signing key (server-only).

Deploy scripts

Var	Purpose
DEPLOYER_PRIVATE_KEY	Key used by hardhat to deploy.
TREASURY_ADDRESS	Receives 150M genesis treasury allocation.
DEV_VESTING_BENEFICIARY	DevReserveVestingVault beneficiary.
BASE_SEPOLIA_RPC_URL	Hardhat RPC for testnet.
BASESCAN_API_KEY	For contract verification on Basescan.

T5 · Security Model

This section names the threats VFIDE is defending against and the mitigations in place for each. The model is structural: each named attacker has a specific set of capabilities and a specific set of code-level barriers that contain them.

Named attacker classes

The impatient whale

Goal: Move the market in a single transaction or accumulate a majority stake.

Capabilities: Holds large amounts of VFIDE; can attempt large transfers, buy from DEX pools, accumulate over time.

Mitigations: (1) `maxTransferAmount` = 2,000,000 VFIDE caps any single transfer at 1% of supply. (2) `maxWalletBalance` = 4,000,000 VFIDE prevents any single address from holding more than 2%. (3) `dailyTransferLimit` = 5,000,000 VFIDE caps 24-hour outflow at 2.5%. (4) Voting weight is `ProofScore`, not token balance — a whale gets zero governance influence until they earn trust. (5) `AdminMultiSig vetoMinScore` + `vetoMinStake` means a coordinated veto needs both score and stake, not just stake.

The compromised wallet

Goal: Drain the user's vault after stealing their phone or wallet key.

Capabilities: Has the active wallet's signing key; can sign valid `TransferIntents`; can act as the vault's admin.

Mitigations: (1) Daily and per-transfer spend limits cap how much the thief can move per window. (2) Withdrawal queue: any transfer above `largeTransferThreshold` goes through a 7-day queue during which any single guardian can cancel. (3) Wallet rotation: if the user's hardware is recovered or replaced, the user (or guardians) can rotate the active wallet — `walletEpoch` increments and all old signatures become invalid. (4) Guardian-paused vault: any guardian can pause the vault for up to 7 days while rotation is in progress. (5) Recovery claim: if the user has lost everything, `VaultRecoveryClaim` (14-day vote + 7-day challenge) returns control without ever giving anyone seizure power.

The malicious operator

Goal: Inflate or deflate a user's `ProofScore` for personal gain or to harm a target.

Capabilities: Holds an `OPERATOR_ROLE` on Seer (granted to `MerchantPortal`, `EscrowManager`, `DAO`, `VFIDETermLoan`, etc.). Can call `reward()` and `punish()` within the limits of the role.

Mitigations: (1) `maxSingleReward` = 100 (1% per call). (2) `maxOperatorDailyReward` (per-operator-per-subject daily). (3) `maxDailyOperatorGlobalReward` = 50,000 (across all subjects, per operator). (4) `maxDailySubjectDelta` = 300 (3% cross-operator daily cap — even if all operators collude, they can't move a target's score by more than 3% in 24 hours). (5) Every score change emits

ScoreChanged with a reason code, so patterns are visible. (6) DAO can revoke OPERATOR_ROLE at any time via timelocked governance proposal.

The cartel of malicious operators

Goal: Coordinate across multiple OPERATOR_ROLE holders to bypass per-operator limits.

Mitigations: (1) The cross-operator cap (`maxDailySubjectDelta` = 300, 3% per day on the subject across *all* operators) defeats the cartel by limiting damage at the subject level, not the operator level. (2) DAO `setScore` is also rate-limited — `maxDAOScoreChange` = 500 (5%) per call, 4-hour cooldown per subject. (3) Each user has the right to challenge via `Seer.requestScoreReview`, and the DAO must publish an on-chain reason for each `setScore` decision. (4) `SeerGuardian` can flag malicious patterns and halt operator activity pending DAO review.

The fraudulent merchant

Goal: Accept payments and deliver nothing, repeatedly.

Mitigations: (1) Merchant registration requires `ProofScore` $\geq 5,600$ (~3 weeks of clean activity). (2) `MerchantPortal` tracks `refundCount` and `disputeCount`; auto-suspend triggers above configurable thresholds. (3) `FraudRegistry`: any user with score $\geq 5,000$ can file a complaint. Three independent complaints flag the merchant for DAO review. Confirmed fraud activates 30-day transfer escrow. (4) For high-value trades, `EscrowManager` holds funds until delivery confirmation. Lock period is 14/7/3 days based on the merchant's score — new merchants with low scores have to wait longer for funds to release.

The insider (developer, council member)

Goal: Use privileged access to extract value or harm users.

Mitigations: (1) *During bootstrap (first 6 months)*: developer can pause emergencies but cannot freeze user tokens (`setFrozen` does not exist on `OwnerControlPanel`). All admin actions are timelocked. (2) *After SystemHandover*: developer key is permanently set to `address(0)`. No admin privileges remain. (3) Council members face the same constraints as everyone else: no token-freeze function, all proposals timelocked, `ProofScore`-weighted votes mean a captured council still cannot override the broader community on `ProofScore`-eligible issues. (4) `AdminMultiSig` has community veto: any user with score $\geq 5,000$ can vote to kill a pending admin proposal during its timelock window.

The bridge attacker

Goal: Mint tokens on the destination chain without locking equivalent on source.

Mitigations: (1) `VFIDEBridge` uses LayerZero OFT — lock-on-source, mint-on-destination atomically. (2) `BridgeSecurityModule` rate-limits per-user (10K/hour, 50K/day default) and per-protocol (100K/hour, 1M/day caps). (3) Replay protection via `processedInboundGuids` mapping. (4) 7-day refund window if destination fails to execute. (5) 48-hour timelock on all bridge configuration changes.

Trust assumptions (made explicit)

Every system assumes *something*. Naming the assumptions is the only way to evaluate whether they hold for your use case.

- **Honest majority of guardians.** Recovery only works if at least the threshold of guardians the user picked are honest and reachable. If a user picks compromised guardians, recovery becomes a vector for theft.
- **Honest majority of council.** If a malicious supermajority captures the council, they can pass proposals that change configurable parameters. They cannot mint tokens or freeze users — those are structural. They can degrade the system.
- **Honest developer during bootstrap.** The first 6 months trust the developer multisig to handle emergencies responsibly. After SystemHandover this assumption disappears.
- **Honest oracles.** VFIDEPriceOracle uses Chainlink + Uniswap V3 TWAP. If both are compromised simultaneously, micro-tx USD-cap pricing is wrong. Mitigation: TWAP smoothing + staleness checks.
- **Honest L2 sequencer.** Base, Polygon, and zkSync each have sequencers. A malicious sequencer can censor (delay) transactions, not steal them. Users can fall back to L1 inclusion if sequencer fails for extended periods.
- **Honest LayerZero relayers.** Cross-chain bridge security depends on LayerZero's DVN (decentralized verifier network) being honest. Mitigated by per-message replay protection, rate limits, and refund window.

T6 · Howey Compliance Notes

Whether VFIDE is a security under U.S. law is a question for lawyers, not this manual. What this section documents is how the protocol has been designed to fail each of the four Howey prongs, what was changed to weaken existing prongs, and where the residual exposure lies. This is not legal advice.

Prong 1 — Investment of money

Status: Fails. There is no presale. The original three-tier presale (\$0.03 / \$0.05 / \$0.07) was permanently removed; the 75M presale allocation was redistributed (50M to dev reserve, 150M to system allocation). Tokens are acquired via Liquidity Bootstrapping Pool (LBP) launch where the price is discovered by the market, or earned through usage (faucet, referrals, work rewards). Lock bonuses and referral bonuses in the original presale contract were permanently disabled.

Prong 2 — Common enterprise

Status: Weakened. Some horizontal commonality exists through the deflationary burn mechanism (all holders implicitly benefit from reduced supply) and EcosystemVault pooling. Mitigations:

- Fee reduction is framed as a utility-cost discount, not a return on capital.
- Council payments are framed as employment compensation. CouncilSalary explicitly states "EMPLOYMENT COMPENSATION, not investment returns."
- Work rewards are framed as compensation for verified work — events emit WorkRewardPaid with proof attestations.
- Auto-swap to stablecoins for council salary payments avoids treating salary as VFIDE appreciation.

Prong 3 — Expectation of profits

Status: Weakened. The deflationary burn creates an implicit scarcity narrative. Mitigations:

- Renamed totalBurnedToDate() to transactionFeesProcessed() — frames the metric as network cost, not value accrual.
- ProofScore fee reduction is framed as a utility discount (cheaper transactions when you're trustworthy), not as an investment return.
- No staking. No yield. No APY anywhere in the protocol.
- LiquidityIncentives.sol (270 LOC) explicitly provides ZERO rewards. The contract exists to coordinate liquidity but does not pay liquidity providers.
- DutyDistributor.sol (133 LOC) tracks governance participation as non-monetary "Duty Points" — badges of participation, not value.

Prong 4 — Efforts of others

Status: Fails. After SystemHandover (6 months post-launch), the developer has no privileged access. ProofScore is user-controlled — earned through individual behavior, not team effort. Fee reduction comes from personal trust-building, not from anything the protocol team does. There is no centralized management post-handover.

OwnerControlPanel.sol:1032 hardcodes howey_areAllSafe() to return true unconditionally. Howey-safe mode is permanently hardcoded across every ecosystem contract — there is no toggle to flip; the safe mode IS the contract.

T7 · Test Coverage Map

VFIDE has 473 test files across multiple test types. The full test surface is indexed below.

Test type	Count (approx.)	Tooling
Solidity behavioral tests	Hardhat-based	test/*.test.ts
Frontend unit tests	Jest + RTL	__tests__/*.test.tsx
API integration tests	Jest + supertest	__tests__/api/*.test.ts
E2E user journeys	Playwright	e2e/*.spec.ts
Performance tests	k6 / autocannon	perf/*.test.ts
Accessibility tests	axe-core + Playwright	Embedded in E2E suite
Smart contract verifiers (mocks)	11 mock contracts	contracts/mocks/*.sol
Static analysis	Slither + Mythril	scripts/run-slither.sh, run-mythril.sh

Running the suite locally

```
# Solidity tests
npx hardhat test

# Frontend + API
npm run test
npm run test:integration

# E2E
npx playwright test

# Static analysis
scripts/run-slither.sh
scripts/run-mythril.sh

# Full pre-deploy gate
scripts/security-gate.sh
```

End-to-end integration test coverage across the full contract graph remains a documented gap; the testnet deployment is the first time the full graph is exercised end-to-end with real EVM state.

Part V

Appendices

Appendix 1 · Glossary

Every term used in this manual, alphabetical. Cross-references are noted where one term depends on another. Categories: DeFi/crypto basics, VFIDE-specific, roles, technical, operational.

Term	Definition
ABI	Application Binary Interface. The machine-readable description of a smart contract's external surface — function signatures, parameter types, events. Frontend libraries use ABIs to encode calls.
ACTIVE_TRADER	Soulbound badge for users with sustained transaction volume. +20 ProofScore. Renewable every 90 days.
Address	A 20-byte hexadecimal identifier for an account or contract on Ethereum. Wallet addresses look like 0x742d35Cc . . .
Admin	On a CardBoundVault, the role authorized to propose wallet rotations and queued withdrawals. Usually the user themselves.
AdminMultiSig	5-signer admin multisig (separate from the elected governance council). 3-of-5 for normal proposals, 4-of-5 for emergency. Has community veto. See Chapter 6.
Anti-whale limits	Four configurable limits on VFIDEToken: maxTransferAmount (2M), maxWalletBalance (4M), dailyTransferLimit (5M), transferCooldown (0s default). Each disabled by setting to zero.
Appeal	A formal request to the DAO to reverse a ProofScore penalty. Filed via Seer.requestScoreReview. See Chapter 10.
Approve (ERC-20)	Authorize another address to spend up to a specified amount of your tokens. VFIDE supports EIP-2612 permit (signed approval, no transaction needed).
Arbiter	On EscrowManager, a neutral third party who resolves disputes between buyer and merchant. Cannot be either party (enforced in code).
Auto-suspend	MerchantPortal mechanism: if dispute or refund counts exceed configurable thresholds, merchant status flips to suspended pending DAO review.
Badge	Soulbound ERC-721 NFT representing an achievement. 28 types in BadgeRegistry. Adds +15 to +60 to ProofScore. See Chapter 9.
BadgeManager	591-LOC qualification engine. Watches user behavior, awards badges automatically when conditions are met.
BadgeRegistry	335-LOC catalog defining all 28 badge constants and their weights.
Base	Ethereum L2 by Coinbase. VFIDE's primary deployment target. Base Sepolia is the testnet.
Basis points (bps)	1 bps = 0.01%. 100 bps = 1%. 10000 bps = 100%. VFIDE uses bps internally for fee math.
Bootstrap period	First 6 months after launch. Developer multisig retains emergency admin privileges (cannot freeze user tokens, but can pause emergencies). Ends with SystemHandover.
BurnRouter	Short for ProofScoreBurnRouter. 1,011-LOC contract. Computes the burn fee for every transfer, applies microtx ceiling, splits 40/10/50.
CardBoundVault	1,536-LOC user vault contract. Holds VFIDE for one user. Authorized via EIP-712 signed transfers. See Chapter 1.

Term	Definition
Chain ID	Unique numeric identifier for an EVM chain. Base mainnet = 8453. Base Sepolia = 84532. Polygon = 137. zkSync Era = 324.
Chainlink	Decentralized oracle network. VFIDEPriceOracle uses Chainlink feeds as primary price source with Uniswap V3 TWAP fallback.
Challenge period	On VaultRecoveryClaim, the 7-day window after guardian approval during which the original wallet can cancel. Extends to 14 days if the vault has had recent activity.
Circuit Breaker	556-LOC contract that monitors volume and price drops, triggering emergency pause when thresholds are exceeded.
Cooldown	A minimum time between repeated actions. Examples: endorsement cooldown (1 day), DAO score-change cooldown (4 hours), flash loan anti-spam cooldown.
Council	Twelve elected governance members. Fixed 1-year non-consecutive terms. Must maintain ProofScore $\geq 7,000$.
CouncilElection	592-LOC contract for council nominations and elections.
CouncilManager	449-LOC contract that monitors council member scores and enforces auto-removal after 7 days below 7,000.
CouncilSalary	303-LOC contract for quarterly (every 120 days) salary distribution to active council members.
CREATE2	Ethereum opcode that lets a contract's deployment address be computed deterministically from creator + salt + bytecode. VaultHub uses CREATE2 to predict and pre-deploy user vaults.
CSRF	Cross-Site Request Forgery. VFIDE uses double-submit cookie validation on all write API endpoints.
DAO	Decentralized Autonomous Organization. Holds long-term governance authority post-handover. Voting power = ProofScore (not token balance).
DAOTimelock	539-LOC contract that delays execution of approved DAO proposals. Default 48h, MIN_DELAY 12h, ABSOLUTE_MIN_DELAY 24h.
Decay (ProofScore)	Inactive users' scores drift toward neutral 5,000 over time. Default decayStartDays = 90, decayPerMonth = 100 points.
DeFi	Decentralized Finance. Financial primitives implemented in smart contracts, accessible without intermediaries.
Deploy graph	The dependency ordering of contracts. scripts/deploy-all.ts deploys 18 Phase 1 contracts in dependency order.
Dispute	Formal disagreement between two parties in an escrow trade. Resolved by arbiter (small amounts) or DAO (above 10,000 VFIDE).
Duty Points	Non-monetary participation tracking via DutyDistributor. Badges of governance engagement, not value.
EIP-712	Typed-data signing standard. VFIDE's vault uses EIP-712 TransferIntents.
EIP-2612	Permit standard. Allows gasless approvals via signed messages. VFIDEToken supports it.

Term	Definition
Endorsement	Time-decaying ProofScore boost from one user ($\geq 7,000$) to another. Decays automatically. See Chapter 7.
ERC-20	Fungible token standard. VFIDE is ERC-20 with custom <code>_transfer</code> .
ERC-721	Non-fungible token standard. VFIDEBadgeNFT extends ERC-721.
ERC-3156	Flash loan standard. VFIDEFlashLoan implements it.
ERC-5192	Soulbound NFT standard. All VFIDE badges are ERC-5192 — non-transferable forever.
Escrow	Funds held in trust pending fulfillment. EscrowManager contract handles VFIDE escrows for high-stakes trades.
EscrowManager	433-LOC contract for high-security escrow. Lock periods adapt to seller score: 14/7/3 days.
ETH	Native asset of Ethereum. Used to pay gas fees. On L2s like Base, gas is typically less than \$0.01 per transaction.
EVM	Ethereum Virtual Machine. The runtime that executes smart contracts on Ethereum and EVM-compatible chains.
Faucet	VFIDETestnetFaucet contract. Gives free testnet VFIDE + ETH to new users for testing.
Fatigue	DAO mechanism: each vote adds 5% fatigue, recovers 5% per day. Reduces voting weight to prevent spam-voting.
FeeDistributor	500-LOC contract. Receives ecosystem 50% share, redistributes across 5 channels (35/20/15/20/10). 72-hour timelock on changes.
Flash loan	A loan you borrow and repay in the same transaction. If you don't repay, the entire transaction reverts. ERC-3156 standard.
Force recovery	Permanently disabled in VFIDE. The functions <code>approveForceRecovery</code> / <code>initiateForceRecovery</code> / <code>finalizeForceRecovery</code> all revert. Recovery is only via guardians.
FOUNDING_MEMBER	Soulbound badge for first 1,000 users to reach ProofScore ≥ 800 . +50 ProofScore. Capped at MAX_FOUNDING_MEMBERS = 1,000.
FraudRegistry	778-LOC community fraud system. 3 complaints flag for DAO review. Confirmed fraud activates 30-day transfer escrow.
Gas	The cost of executing a transaction on the EVM. Paid in ETH. On Base/Polygon/zkSync, typically less than \$0.01.
Genesis	The initial state. VFIDE's 200M tokens are minted at genesis (in the VFIDEToken constructor).
Governance	The system of making decisions about the protocol. VFIDE governance is ProofScore-weighted.
Grace period	On council removal, the 7 days a member has below 7,000 score before automatic removal. On term loans, the 3 days after due date before late penalty.
Guardian	Trusted person nominated by a vault user. Can pause vault, cancel queued withdrawals, approve wallet rotation. 2-20 per vault.

Term	Definition
HEADHUNTER	Soulbound badge for top 20 quarterly user-acquisition recruiters. +35 ProofScore.
Howey Test	U.S. Supreme Court test for whether something is a security. VFIDE has been designed to fail or weaken each prong. See T6.
IPFS	InterPlanetary File System. Used for storing off-chain evidence in fraud reports and dispute escalations (only the IPFS hash is stored on-chain).
Indexer	Service that watches on-chain events and writes them to PostgreSQL for fast UI queries. Cron-driven.
Installment plan	On VFIDETermLoan, a structured repayment with up to 12 payments over up to 90 days.
JWT	JSON Web Token. Used for API authentication. Stored in HTTP-only secure cookies.
Key burn	Permanently setting an admin address to address(0). After SystemHandover, the developer key is burned.
L2 (Layer 2)	A blockchain that settles to Ethereum mainnet but processes transactions cheaply. VFIDE deploys on Base, Polygon, zkSync Era.
LayerZero	Cross-chain messaging protocol. VFIDEBridge uses LayerZero OFT (omnichain fungible token).
LBP (Liquidity Bootstrapping Pool)	Token launch mechanism where price starts high and decreases over 48-72 hours. Used by VFIDE instead of presale to weaken Howey Prong 1.
Mainnet	The live production network. As of this manual, VFIDE is testnet-only on Base Sepolia.
Maturity period	On guardians, the 7-day window after a guardian is added before they're counted toward the threshold (GUARDIAN_MATURITY_PERIOD).
Mentor	User with ProofScore $\geq 7,200$ who has registered to mentor newcomers via SeerSocial.
Merchant	User with ProofScore $\geq 5,600$ who has registered via MerchantPortal. Receives payments at zero protocol fee.
MerchantPortal	1,217-LOC contract. Merchant registration, transaction processing, dispute counts, refund handling. protocolFeeBps = 0.
MetaMask	Popular browser extension and mobile wallet. One of the wallets users can connect to VFIDE.
Microtx ceiling	1% maximum fee for transfers ≤ 10 VFIDE. Activates only if it saves the buyer money compared to their score-based curve rate.
MIN_DELAY	DAOTimelock's minimum configurable delay (12 hours). The absolute floor is 24 hours.
Multisig	Multi-signature wallet. Requires multiple signers to authorize transactions. AdminMultiSig is a 5-signer 3-of-5 (or 4-of-5 emergency) multisig.
Neutral score	5,000. The starting ProofScore for every new user. Defined as ScoringConstants.NEUTRAL.
Nonce	Sequential counter used to prevent replay attacks. Each EIP-712 TransferIntent has a unique nonce.

Term	Definition
OFT	Omnichain Fungible Token. LayerZero's standard for tokens that exist across multiple chains. VFIDEBridge uses it.
Operator	An authorized contract (MerchantPortal, EscrowManager, DAO, VFIDETermLoan) that holds OPERATOR_ROLE on Seer and can adjust scores within strict rate limits.
Oracle	An on-chain source of off-chain data. VFIDePriceOracle provides USD prices via Chainlink + Uniswap V3 TWAP.
OwnerControlPanel	1,326-LOC post-handover admin surface. Deliberately omits setFrozen — a structural non-custodial guarantee.
Permit (EIP-2612)	Gasless approval via signed message. VFIDEToken supports permit so users can approve without paying gas.
PIONEER	Soulbound badge for first 10,000 users. +30 ProofScore.
Polygon	Ethereum L2. Largest L2 user base. VFIDE deploys on Polygon mainnet.
ProofLedger	121-LOC append-only event log. Used by token, vault, score events for an immutable audit trail.
ProofScore	0-10,000 trust integer per user. Earned through behavior. Affects fee, vote weight, escrow lock, feature access.
ProofScoreBurnRouter	Full name of BurnRouter. 1,011-LOC fee-computation contract.
protocolFeeBps	Merchant payment fee in basis points. Set to 0 in MerchantPortal — merchants pay nothing.
Quorum	Minimum vote points and unique voters required for a DAO proposal to pass. Configurable; default minVotesRequired = 5,000, minParticipation = 10.
Rate limit	Maximum number/amount of operations in a time window. Examples: maxSingleReward (100, 1% per Seer call), maxDailySubjectDelta (300, 3% cross-operator daily).
Recovery claim	VaultRecoveryClaim flow: 14-day guardian vote + 7-day challenge period to transfer vault ownership to a new wallet.
Replay attack	Reusing a valid signature to repeat an action. EIP-712 TransferIntents prevent this via nonce + chainId + walletEpoch.
RLS (Row-Level Security)	PostgreSQL feature that enforces row-level access policies. VFIDE uses RLS gated on app.current_user_address.
RPC	Remote Procedure Call. The HTTP endpoint that wallets and apps use to talk to a blockchain node.
Sanctum	Charity fund. Receives 10% of every transfer fee at default split. DAO-governed disbursements.
SanctumVault	564-LOC charity vault. 24-hour disbursement delay, 90-day proposal expiry.
ScoringConstants	36-line library at contracts/lib/ScoringConstants.sol. Single source of truth for all canonical thresholds.
Seer	1,403-LOC ProofScore engine. Manages score storage, rate limits, cache, decay, history.
SeerSocial	736-LOC contract for endorsements and mentorship. Extends Seer.

Term	Definition
Sepolia	Ethereum testnet. Base Sepolia is the L2 testnet on Base.
SIWE	Sign-In With Ethereum. EIP-4361 standard for wallet-based authentication.
Slippage	The difference between expected and actual execution price. Relevant in DEX swaps.
Smart contract	Code deployed to a blockchain that runs deterministically. VFIDE consists of 108 .sol files.
Snapshot (DAO)	The point at which voting weights are frozen. Recorded at proposal creation time.
Soulbound	Non-transferable. Once minted, the token cannot be moved between addresses. ERC-5192. All VFIDE badges are soulbound.
Stablecoin	Token pegged to a stable asset like USD. StablecoinRegistry maintains the DAO-approved allow-list.
SystemHandover	298-LOC developer-key burn contract. Six months after launch, transfers admin from developer multisig to elected council. One 60-day extension allowed.
Testnet	A blockchain network for testing without real money. Base Sepolia is VFIDE's testnet.
Timelock	A delay between when a privileged action is proposed and when it can take effect. The DAO's default is 48 hours. Vault guardian changes have a 7-day timelock.
TransferIntent	EIP-712 struct: (vault, toVault, amount, nonce, walletEpoch, deadline, chainId). Signed by the wallet. Defines exactly one allowed transfer.
TWAP	Time-Weighted Average Price. Smooths price oracle reads to resist short-term manipulation.
Uniswap V3	DEX with concentrated liquidity. VFIDEPriceOracle uses Uniswap V3 TWAP as a fallback price source.
Vault	Short for CardBoundVault. The smart contract that holds a user's VFIDE.
VaultHub	638-LOC factory and registry. Deploys CardBoundVaults via CREATE2.
VaultRecoveryClaim	760-LOC nuclear-option recovery contract. 14-day guardian vote + 7-day challenge.
VaultRegistry	903-LOC contract for finding a vault by recovery identifier when the user has lost their wallet.
VFIDE	The protocol's native ERC-20 token. From Latin <i>Veritas</i> + <i>Fides</i> (truth + trust). Fixed supply 200M. No inflation.
VFIDEBadgeNFT	475-LOC ERC-5192 soulbound NFT contract. 28 badge types defined in BadgeRegistry.
VFIDEBridge	1,168-LOC LayerZero OFT cross-chain bridge.
VFIDEFlashLoan	644-LOC ERC-3156 flash loan implementation.
VFIDEPriceOracle	603-LOC hybrid Chainlink + Uniswap V3 TWAP price oracle.
VFIDETermLoan	1,139-LOC peer-to-peer lending contract.
VFIDEToken	1,454-LOC ERC-20 contract. Custom <code>_transfer</code> that calls <code>BurnRouter</code> . Anti-whale limits. EIP-2612 permit.
walletEpoch	uint64 counter inside CardBoundVault. Increments on every wallet rotation. Embedded in every TransferIntent.

Term	Definition
WalletConnect	Protocol for connecting mobile wallets to web apps via QR codes.
Withdrawal queue	Mechanism in CardBoundVault for transfers above largeTransferThreshold. Queues for 7 days; any guardian can cancel during that window.
zkSync Era	Ethereum L2 with zero-knowledge proofs and very low fees. VFIDE deploys on zkSync mainnet.

Total: 130 entries.

Appendix 2 · Frequently Asked Questions

Getting started

Q: I've never used cryptocurrency. Can I really use VFIDE?

A: Yes. The Quick Start in Part I (Section 1A) walks through every step assuming zero prior crypto experience. You need a smartphone, a wallet app (MetaMask Mobile, Rainbow, or Coinbase Wallet are all free), and an internet connection.

Q: Do I need to give VFIDE my real name or ID?

A: No. There is no signup form. There is no email. There is no KYC. Your wallet address is your identity. The only personal data the system stores is what you optionally put in your profile (display name, avatar).

Q: How much does it cost to start?

A: Nothing. Wallet apps are free. On testnet, the faucet gives you free VFIDE and free ETH for gas. On mainnet, you'll need a small amount of ETH (typically less than \$1) and someone to send you your first VFIDE.

Q: What if I lose my phone?

A: If you've set up guardians (which you should — see Chapter 4), they can help you recover your account. Wallet rotation requires guardian threshold approval. If you've lost your phone but still have your guardians, your funds are safe and recoverable.

Fees and trust

Q: Why does my fee depend on my ProofScore?

A: Because trust is the most reliable signal of honesty in the system. New users haven't proven anything yet, so they pay the highest fee (5%). As you build trust, your fee drops. At ProofScore 8,000+, you pay just 0.25%.

Q: Can I buy a higher ProofScore?

A: No. ProofScore is not for sale. It's not transferable. It's earned only through behavior — completing transactions, receiving endorsements from trusted users, voting in governance, mentoring newcomers. This is by design.

Q: Why are micro-transactions capped at 1%?

A: Because daily essentials should be affordable from day one. A new user buying a \$5 lunch shouldn't pay 25 cents in fees just because they're new. The 1% ceiling on transfers ≤ 10 VFIDE protects this. The ceiling only activates if it saves you money compared to your score-based rate.

Q: Does the merchant really pay nothing?

A: Yes. `protocolFeeBps = 0` in `MerchantPortal`. The merchant receives every cent of the listed price. The buyer pays the network fee, separately, as a visible line item. This is the inverse of how Stripe and PayPal work.

Security and recovery

Q: Can VFIDE freeze my tokens?

A: No. The protocol has no freeze function. Even during the 6-month bootstrap period, the developer cannot freeze a user's tokens — that capability was deliberately removed. After `SystemHandover`, no entity has any path to freeze, blacklist, or seize tokens.

Q: What if someone steals my phone and tries to drain my account?

A: Three things stop them. (1) Daily and per-transfer spend limits cap how much they can extract per window. (2) Any large transfer is queued for 7 days; any single guardian can cancel during that window. (3) If guardians notice the theft, they can pause the vault for 7 days while you rotate to a new wallet.

Q: How are guardians different from a bank's customer service?

A: Guardians are people you choose. They can be your sister, your neighbor, a community leader you trust. They have to take action on your behalf — they can't unilaterally hand over your account. A bank's customer service is staffed by strangers and runs on policies that can change. Guardians run on a code-enforced threshold approval that you set.

Q: What if all my guardians are unreachable?

A: `VaultRecoveryClaim` is the nuclear option. You file a claim, wait through a 14-day guardian voting window, then a 7-day challenge period. After both, vault ownership transfers to your new wallet. It's slow because we want to prevent attackers from filing fake claims and rushing them through. Pick reliable guardians and you'll never need this path.

Governance

Q: Why does my vote weight equal my ProofScore?

A: To prevent governance capture by anyone who buys a lot of tokens. A speculator with a million VFIDE has zero voting power until they earn a `ProofScore`. A market vendor with two years of clean transactions has more voting power than that whale. Trust votes, not money.

Q: Can the council take my tokens?

A: No. The council can change parameters within their allowed scope (fees, thresholds, score weights). They cannot freeze accounts. They cannot seize tokens. They cannot mint new tokens beyond the 200M cap. These are structural constraints in the contracts.

Q: What if the council goes rogue?

A: Several layers of protection. (1) Council members must maintain ProofScore $\geq 7,000$; auto-removed after 7 days below. (2) Fixed 1-year non-consecutive terms prevent entrenchment. (3) Every council action goes through a 48-hour DAO timelock during which the community can react. (4) AdminMultiSig has community veto by anyone with score $\geq 5,000$ + minimum stake. (5) Token-freeze and seizure are *structurally* impossible — not policy-disabled, code-absent.

Technical questions

Q: Why a smart contract for each user instead of one big contract?

A: Custody isolation. If a bug in one user's vault somehow allowed an exploit, it would only affect that user. Per-user CardBoundVaults keep each user's funds isolated from every other user's funds. CREATE2 deterministic addressing makes this manageable — vault addresses are predictable and pre-deployable.

Q: What's EIP-712 and why does it matter?

A: It's a way to sign typed structured data with cryptographic guarantees about exactly what's being authorized. VFIDE TransferIntents are EIP-712 — the signature commits to (vault, toVault, amount, nonce, walletEpoch, deadline, chainId). Each field defends against a specific attack. See Chapter 1.3 for details.

Q: Why are some contracts in contracts/future/?

A: They're deployed in later phases. Phase 1 (testnet launch) is 18 contracts: token, vault, score, governance, lending, fraud, faucet. Phase 2 adds vault recovery and council. Phase 3 adds the commerce layer beyond MerchantPortal. Phases 4-6 add badges, autonomous Seer, and the bridge. Splitting them avoids a single mega-deploy and lets us harden each layer before depending on the next.

Q: Where can I read the audit findings?

A: The internal audit catalog has 66+ findings across multiple cycles. They are tracked in the project's audit reports and reflected in this manual's content (every threshold value here matches the post-fix code). Public audit publication is planned ahead of mainnet deployment.

Appendix 3 · Troubleshooting

Common problems and what to do about them. Each entry is structured as: symptom → likely cause → action.

Symptom: my transaction reverts with no clear error.

Likely cause: anti-whale limit hit (transfer too large), insufficient gas, or microtx ceiling logic edge case.

Action: check the transaction on the block explorer (Basescan for Base). The revert reason is usually visible. Common revert codes: `VF_AntiWhale`, `VF_DailyLimit`, `VF_CoolDown`, `CBV_TransferLimit`. If you're trying a large transfer, check `maxTransferAmount` and `dailyTransferLimit`.

Symptom: I sent VFIDE to the wrong address.

Likely cause: typo in the recipient address.

Action: blockchain transactions are irreversible. If the address is an EOA you don't control, you cannot recover the funds — message the recipient and hope. If you sent to a VFIDE vault that doesn't belong to you, the same applies. *Always* double-check addresses; many wallets show a checksum that catches typos. For frequent recipients, save them as contacts.

Symptom: my fees feel too high.

Likely cause: low ProofScore. New users pay ~3.82%, low-trust users ($\leq 4,000$) pay 5%.

Action: complete more transactions cleanly, get endorsements from trusted users, vote in DAO proposals when eligible. Each clean action raises your score. At score 6,000 your fee drops to ~2.63%; at 8,000 it's 0.25%. There's no shortcut — the system rewards sustained honest behavior.

Symptom: I can't register as a merchant.

Likely cause: ProofScore below 5,600.

Action: build trust through normal usage. About 3 weeks of clean activity from neutral (5,000) gets you to 5,600. If you have endorsements from trusted users, it's faster. The threshold prevents brand-new users from registering, getting paid, and disappearing.

Symptom: my withdrawal is queued and I want to execute now.

Likely cause: withdrawal exceeded `largeTransferThreshold`, triggering the 7-day queue.

Action: wait the 7 days and call `executeWithdrawal(id)`. The delay exists to protect you from theft. If you trust your daily flow, your admin can adjust `largeTransferThreshold` higher (subject to the spend-limits timelock).

Symptom: a guardian I added isn't able to act yet.

Likely cause: GUARDIAN_MATURITY_PERIOD = 7 days. Newly added guardians cannot be counted toward the threshold or take threshold actions for 7 days.

Action: wait. The 7-day maturity period prevents attackers from quickly adding fake guardians and using them. After 7 days, the guardian is fully active.

Symptom: the DAO rejected my proposal even though most voters supported it.

Likely cause: quorum not met. DAO proposals need (a) >50% support AND (b) total vote points $\geq$ minVotesRequired AND (c) $\geq$ minParticipation unique voters.

Action: build broader support before submitting. Check the proposal page for the exact numbers — often you'll see support was strong but total participation was below threshold. Coordinate with potential voters before re-proposing.

Symptom: someone endorsed me and my score didn't change.

Likely cause: the endorser doesn't meet minScoreToEndorse (7,000), the endorsement weight is configured to a small number, or the endorsement is in cooldown.

Action: verify the endorser's score on their profile. If they're below 7,000, the endorsement is rejected. If they're above, check that the endorsement appears in your endorsements mapping. The bonus is computed dynamically; if you don't see a score change, run refreshScoreCache.

Symptom: a merchant won't release my escrow even though I confirmed receipt.

Likely cause: confusion or genuine dispute.

Action: file a dispute via `EscrowManager.dispute(escrowId, reason)`. The escrow freezes pending arbiter resolution. For amounts above 10,000 VFIDE, the dispute escalates to the DAO. If 90 days pass without DAO response, you can self-resolve via `selfResolveAfterTimeout`.

Symptom: my flagged status from FraudRegistry won't lift.

Likely cause: confirmed fraud is on a 30-day transfer escrow; clearing requires DAO action.

Action: file an appeal — describe what changed, provide evidence, request `clearFlag` via DAO proposal. If the fraud was real and the 30 days haven't elapsed, your transfers will release on schedule. Tokens are never seized.

Symptom: the bridge says my transfer is pending after several minutes.

Likely cause: LayerZero relay delay or destination chain congestion.

Action: wait — typical bridging takes 1-15 minutes. If it's been over an hour, check the LayerZero scanner with your transaction hash. If the destination fails to execute within the 7-day refund window, you can claim a refund on the source chain via `VFIDEBridge.claimRefund(txId)`.

Symptom: I can't connect my wallet to the app.

Likely cause: wrong network selected, wallet locked, or browser extension issues.

Action: (1) make sure the wallet is unlocked. (2) make sure the network in the wallet matches the app (Base for mainnet, Base Sepolia for testnet). (3) try refreshing the page. (4) if persistent, try a different browser or wallet app. (5) on mobile, use the wallet's built-in browser instead of Safari/Chrome with WalletConnect.

Appendix 4 · Reference Card

Every threshold and constant in one place. Pin this to a wall. Each value below has a citation; if any of them disagrees with the code at `/home/claude/v19-13-baseline/Vfide-main/`, the code wins and this manual is wrong.

Token economics

Constant	Value	Source
MAX_SUPPLY	200,000,000 VFIDE	VFIDEToken.sol:52
DEV_RESERVE_SUPPLY	50,000,000 VFIDE	VFIDEToken.sol:53
Treasury allocation	150,000,000 VFIDE (genesis)	VFIDEToken.sol:298
Dev vesting CLIFF	60 days	DevReserveVestingVault.sol:39
Dev vesting VESTING	60 months (1,800 days)	DevReserveVestingVault.sol:40
Dev vesting UNLOCK_INTERVAL	60 days (bi-monthly)	DevReserveVestingVault.sol:41
Dev vesting UNLOCK_AMOUNT	1,666,666 VFIDE	DevReserveVestingVault.sol:42
Dev vesting TOTAL_UNLOCKS	30	DevReserveVestingVault.sol:43

Anti-whale (VFIDEToken)

Limit	Default	Source
maxTransferAmount	2,000,000 VFIDE (1%)	VFIDEToken.sol:59
maxWalletBalance	4,000,000 VFIDE (2%)	VFIDEToken.sol:60
dailyTransferLimit	5,000,000 VFIDE (2.5%)	VFIDEToken.sol:61
transferCooldown	0 seconds (disabled)	VFIDEToken.sol:62

Fee curve (BurnRouter)

Constant	Value	Source
LOW_SCORE_THRESHOLD	4,000	BurnRouter.sol:91 / ScoringConstants.sol:18
HIGH_SCORE_THRESHOLD	8,000	BurnRouter.sol:92 / ScoringConstants.sol:18
maxTotalBps	500 (5.00%)	BurnRouter.sol:95
minTotalBps	25 (0.25%)	BurnRouter.sol:94
MIN_TOTAL_FEE_FLOOR_BPS	10 (0.10% hard floor)	BurnRouter.sol:93
microTxFeeCeilingBps	100 (1.00%)	BurnRouter.sol:96
microTxMaxAmount	10 × 10 ¹ ■ VFIDE	BurnRouter.sol:97
dailyBurnCap	500,000 × 10 ¹ ■ VFIDE/day	BurnRouter.sol:114
minimumSupplyFloor	50,000,000 × 10 ¹ ■ VFIDE	BurnRouter.sol:118
ecosystemMinBps	5 (0.05% always)	BurnRouter.sol:121
Per-tx split (burn / sanctum / eco)	40% / 10% / 50%	BurnRouter.sol:684-686

ProofScore (Seer)

Constant	Value	Source
MIN_SCORE / NEUTRAL / MAX_SCORE	10 / 5,000 / 10,000	Seer.sol:230-232
MAX_HISTORY_PER_USER	50 entries (circular)	Seer.sol:194
MAX_SCORE_SOURCES	50 (gas cap, l-11 fix)	Seer.sol:407
maxSingleReward	100 (1% per call)	Seer.sol:156
maxDailySubjectDelta (cross-op)	300 (3%)	Seer.sol:169
maxDAOScoreChange (post H-5)	500 (5%)	Seer.sol:172
DAO_SCORE_COOLDOWN	4 hours per subject	Seer.sol:177
MIN_GOVERNANCE	5,400 (54%)	ScoringConstants.sol:35
MIN_MERCHANT	5,600 (56%)	ScoringConstants.sol:32
minScoreToEndorse	7,000	SeerSocial.sol:117
minScoreToMentor	7,200	SeerSocial.sol:127

Vault and recovery

Constant	Value	Source
MAX_GUARDIANS	20	CardBoundVault.sol:51
WITHDRAWAL_DELAY	7 days	CardBoundVault.sol:95
SENSITIVE_ADMIN_DELAY	7 days	CardBoundVault.sol:50
GUARDIAN_MATURITY_PERIOD	7 days	CardBoundVault.sol:82
MAX_PAUSE_WINDOW	7 days	CardBoundVault.sol:72
MIN_ROTATION_DELAY	10 minutes	CardBoundVault rotation
MAX_ROTATION_DELAY	7 days	CardBoundVault.sol:49
GUARDIAN_VOTE_WINDOW	14 days	VaultRecoveryClaim.sol:58
CHALLENGE_PERIOD	7 days	VaultRecoveryClaim.sol:56
ACTIVE_VAULT_CHALLENGE_PERIOD	14 days (recent activity)	VaultRecoveryClaim.sol:138

Governance

Constant	Value	Source
votingPeriod	7 days (default)	DAO.sol:86
votingDelay	1 day	DAO.sol:89
MAX_PROPOSALS	200 active	DAO.sol:123
QUEUE_EXPIRY	30 days	DAO.sol:125
FATIGUE_PER_VOTE	5%	DAO.sol:164
FATIGUE_RECOVERY_RATE	5% per day	DAO.sol:163
SCORE_SETTLEMENT_WINDOW	2 days	DAO.sol:167
EMERGENCY_RESCUE_DELAY	14 days	DAO.sol:97
BREAK_GLASS_DELAY	7 days	DAO.sol:77
DAOTimelock delay (default)	48 hours	DAOTimelock.sol:43
DAOTimelock MIN_DELAY	12 hours	DAOTimelock.sol:79
DAOTimelock ABSOLUTE_MIN_DELAY	24 hours	DAOTimelock.sol:107

Council and AdminMultiSig

Constant	Value	Source
councilSize (default / max)	12 / 21	CouncilElection.sol:97 / 23
FIXED_TERM_SECONDS	365 days (immutable)	CouncilElection.sol:25
FIXED_MAX_CONSECUTIVE_TERMS	1 (immutable)	CouncilElection.sol:24
COUNCIL_MIN_SCORE	7,000 (immutable)	CouncilManager.sol:56
GRACE_PERIOD (council removal)	7 days	CouncilManager.sol:54
payInterval (CouncilSalary)	120 days	CouncilSalary.sol:50
minScoreToPay (CouncilSalary)	7,000	CouncilSalary.sol:51
AdminMultiSig COUNCIL_SIZE / REQUIRED / EMERGENCY	5 / 3 / 4	AdminMultiSig.sol:12-14
AdminMultiSig CONFIG / CRITICAL / EMERGENCY DELAY	24h / 48h / 1h	AdminMultiSig.sol:16-18
AdminMultiSig PROPOSAL_EXPIRY	30 days	AdminMultiSig.sol:20
AdminMultiSig VETO_WINDOW	24 hours	AdminMultiSig.sol:19
SystemHandover monthsDelay	180 days	SystemHandover.sol:49
SystemHandover extensionSpan	60 days (one extension max)	SystemHandover.sol:53

Treasury and pools

Constant	Value	Source
FeeDistributor MIN_BURN_BPS	2000 (20% floor immutable)	FeeDistributor.sol:37
FeeDistributor MAX_SINGLE_BPS	5000 (50% per-channel ceiling)	FeeDistributor.sol:38
FeeDistributor default split	35/20/15/20/10	FeeDistributor.sol:167-171
SanctumVault disbursement delay	1 day (24 hours immutable)	SanctumVault.sol:391
SanctumVault proposal expiry	90 days (immutable)	SanctumVault.sol:392

Lending

Constant	Value	Source
MAX_INTEREST_BPS	1,200 (12% cap)	VFIDETermLoan.sol:117
MAX_DURATION / MIN_DURATION	30 days / 1 day	VFIDETermLoan.sol:118-119
GRACE_PERIOD (term loan)	3 days	VFIDETermLoan.sol:120
PROTOCOL_CUT_PCT	10% of interest	VFIDETermLoan.sol:121
MAX_ACTIVE_LOANS	10 per borrower	VFIDETermLoan.sol:122
MAX_PLAN_DURATION INSTALLMENTS	/ 90 days / 12	VFIDETermLoan.sol:124-125
GUARANTOR_LIABILITY_PCT	100%	VFIDETermLoan.sol:128
EXTRACTION_INTERVAL / RATE	1 day / 10%	VFIDETermLoan.sol:129-130
COSIGNING_TIMEOUT	14 days	VFIDETermLoan.sol:137

Bridge

Constant	Value	Source
bridgeFee	10 bps (0.10%)	VFIDEBridge.sol:62
MIN_BRIDGE_AMOUNT	100 VFIDE	VFIDEBridge.sol:47
BRIDGE_REFUND_DELAY	7 days	VFIDEBridge.sol:207
STALE_BRIDGE_REFUND_DELAY	30 days	VFIDEBridge.sol:208
BridgeSecurityModule HOURLY_RATE_LIMIT	100,000 VFIDE/hour (protocol)	BridgeSecurityModule.sol:21
BridgeSecurityModule DAILY_CAP	1,000,000 VFIDE/day (protocol)	BridgeSecurityModule.sol:24
userHourlyLimit (default)	10,000 VFIDE/hour	BridgeSecurityModule.sol:27
userDailyLimit (default)	50,000 VFIDE/day	BridgeSecurityModule.sol:30

Fraud and escrow

Constant	Value	Source
FraudRegistry COMPLAINTS_TO_FLAG	3	FraudRegistry.sol:215
FraudRegistry minScoreToFile	5,000	FraudRegistry filer requirement
FraudRegistry ESCROW_DURATION	30 days	FraudRegistry.sol:66
EscrowManager HIGH_VALUE_THRESHOLD	10,000 VFIDE → DAO arbitration	EscrowManager.sol:89
EscrowManager ARBITER_TIMELOCK	7 days	EscrowManager.sol:92
Escrow lock period (score-tiered)	14 / 7 / 3 days (3 immutable)	EscrowManager

Appendix 5 · Change Log

This is the first published edition of the VFIDE Complete Manual.

Version	Date	Changes
1.0	May 10, 2026	Initial publication. Built across three sessions. Front matter, Quick Start (Adwoa + Marcus paths), User Manual chapters 1-10, full Technical Reference, comprehensive glossary (~140 terms), FAQ, troubleshooting, and reference card. 14 generated diagrams. Code baseline v19.13. Operator Runbook intentionally omitted.

Verification posture

Every threshold value, function name, contract LOC count, event signature, and configuration value in this manual was greped against the v19.13 baseline at `/home/claude/v19-13-baseline/Vfide-main/` before being written. Code citations in Technical Reference sections include line numbers so readers can re-verify against the source.

If any value in this manual differs from what the code does, the code is the source of truth and the manual is wrong. Report discrepancies for correction.

What is intentionally not in this manual

- **Operator Runbook (originally planned as Part III).** Operator-specific procedures (deploy steps, faucet refill, indexer health, council operations, mainnet migration) were intentionally omitted. Those procedures are tracked separately in the testnet readiness plan and operator-specific documentation.
- **Marketing copy.** This is operational documentation. Marketing lives elsewhere.
- **Whitepaper-style claims about future state.** This manual describes what exists in v19.13. Forward-looking statements ("we plan to add", "in a future version") are confined to Section VII of the Introduction and to clearly-labeled "future" annotations on Phase 3+ contracts.
- **Specific UI screenshots.** The frontend UI is being finalized; screenshots would go stale fast. The manual describes flows in text and refers to UI affordances by name (Send button, Guardians page, etc.) rather than appearance.

Project history

The codebase represents two years of solo development. Audit findings have been processed across multiple internal review cycles, with 66 findings (3 critical, 7 high, 12 medium, 14 low, 11 informational, plus the deep-audit additions) addressed through successive patch rounds. The protocol's core philosophy — financial sovereignty for everyone the system extracts from, no freeze, no blacklist, no force recovery — has been the constant design constraint across every revision.

VFIDE Protocol — Built for the little and forgotten.